INDEX

S. No	Topic	Page No.
	Week 1	
1	Biological Neuron	1
2	From Spring to Winter of AI	8
3	The Deep Revival	18
4	From Cats to Convolutional Neural Networks	25
5	Faster, higher, stronger	29
6	The Curious Case of Sequences	31
7	Beating humans at their own games (literally)	38
8	The Madness (2013-)	41
9	(Need for) Sanity	50
10	Motivation from Biological Neurons	54
11	McCulloch Pitts Neuron, Thresholding Logic	62
12	Perceptrons	73
13	Error and Error Surfaces	82
14	Perceptron Learning Algorithm	85
15	Proof of Convergence of Perceptron Learning Algorithm	98
	Week 2	
16	Deep Learning(CS7015): Linearly Separable Boolean Functions	108
17	Deep Learning(CS7015): Representation Power of a Network of Perceptrons	113
18	Deep Learning(CS7015): Sigmoid Neuron	124
19	Deep Learning(CS7015): A typical Supervised Machine Learning Setup	136
20	Deep Learning(CS7015): Learning Parameters: (Infeasible) guess work	146
21	Deep Learning(CS7015): Learning Parameters: Gradient Descent	157
22	Deep Learning(CS7015): Representation Power of Multilayer Network of Sigmoid Neurons	179
	Week 3	
23	Feedforward Neural Networks (a.k.a multilayered network of neurons)	209
24	Learning Paramters of Feedforward Neural Networks (Intuition)	222
25	Output functions and Loss functions	229
26	Backpropagation (Intuition)	244
27	Backpropagation: Computing Gradients w.r.t. the Output Units	255
28	Backpropagation: Computing Gradients w.r.t. Hidden Units	267
29	Backpropagation: Computing Gradients w.r.t. Parameters	279
30	Backpropagation: Pseudo code	287

31	Derivative of the activation function	292
32	Information content, Entropy & cross entropy	294

Week 4

33	Recap: Learning Parameters: Guess Work, Gradient Descent	312
34	Contours Maps	324
35	Momentum based Gradient Descent	333
36	Nesterov Accelerated Gradient Descent	346
37	Stochastic And Mini-Batch Gradient Descent	355
38	Tips for Adjusting Learning Rate and Momentum	368
39	Line Search	377
40	Gradient Descent with Adaptive Learning Rate	383
41	Bias Correction in Adam	404

Week 5

42	Eigenvalues and Eigenvectors	412
43	Linear Algebra : Basic Definitions	424
44	Eigenvalue Decompositon	434
45	Principal Component Analysis and its Interpretations	442
46	PCA : Interpretation 2	465
47	PCA : Interpretation 3	477
48	PCA : Interpretation 3 (Contd.)	482
49	PCA : Practical Example	484
50	Singular Value Decomposition	491

Week 6

51	Introduction to Autoncoders	510
52	Link between PCA and Autoencoders	543
53	Regularization in autoencoders (Motivation)	556
54	Denoising Autoencoders	564
55	Sparse Autoencoders	582
56	Contractive Autoencoders	590

Week 7

57	Bias and Variance	599
58	Train error vs Test error	608
59	Train error vs Test error (Recap)	624
60	True error and Model complexity	631
61	L2 regularization	639
62	Dataset augmentation	656

63	Parameter sharing and tying	660
64	Adding Noise to the inputs	662
65	Adding Noise to the outputs	667
66	Early stopping	671
67	Ensemble Methods	679
68	Dropout	686

Week 8

69	A quick recap of training deep neural networks	700
70	Unsupervised pre-training	706
71	Better activation functions	726
72	Better initialization strategies	752
73	Batch Normalization	774

Week 9

74	One-hot representations of words	786
75	Distributed Representations of words	793
76	SVD for learning word representations	802
77	SVD for learning word representations (Contd.)	803
78	Continuous bag of words model	814
79	Skip-gram model	839
80	Skip-gram model (Contd.)	846
81	Contrastive estimation	856
82	Hierarchical softmax	861
83	GloVe representations	871
84	Evaluating word representations	877
85	Relation between SVD and Word2Vec	885

Week 10

86	The convolution operation	888
87	Relation between input size, output size and filter size	907
88	Convolutional Neural Networks	919
89	Convolutional Neural Networks (Contd.)	935
90	CNNs (success stories on ImageNet)	951
91	CNNs (success stories on ImageNet) (Contd.)	972
92	Image Classification continued (GoogLeNet and ResNet)	974
93	Visualizing patches which maximally activate a neuron	989
94	Visualizing filters of a CNN	993
95	Occlusion experiments	999

96	Finding influence of input pixels using backpropagation	1003
97	Guided Backpropagation	1008
98	Optimization over images	1012
99	Create images from embeddings	1022
100	Deep Dream	1032
101	Deep Art	1039
102	Fooling Deep Convolutional Neural Networks	1044

Week 11

103	Sequence Learning Problems	1050
104	Recurrent Neural Networks	1057
105	Backpropagation through time	1067
106	The problem of Exploding and Vanishing Gradients	1080
107	Some Gory Details	1089
108	Selective Read, Selective Write, Selective Forget - The Whiteboard Analogy	1095
109	Long Short Term Memory(LSTM) and Gated Recurrent Units(GRUs)	1103
110	How LSTMs avoid the problem of vanishing gradients	1126
111	How LSTMs avoid the problem of vanishing gradients (Contd.)	1133

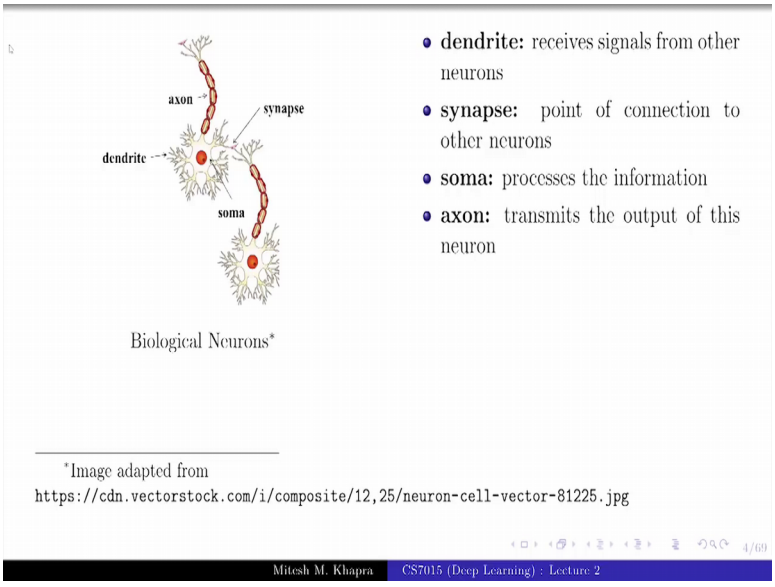
Week 12

112	Introduction to Encoder Decoder Models	1152
113	Applications of Encoder Decoder models	1169
114	Attention Mechanism	1185
115	Attention Mechanism (Contd.)	1205
116	Attention over images	1207
117	Hierarchical Attention	1216

And the question is, why is it called a neuron, where does the inspiration come from. So, we already know that the inspiration comes from biology and more specifically it comes from the brain, because we saw that way back in the 1890s, this term neuron was coined for neural processing units or the cells in our brain.

So, now before we move on to the computational neurons or the artificial neurons, we will just see the biological neurons in a bit more detail and then we will move on from there.

(Refer Slide Time: 01:29)



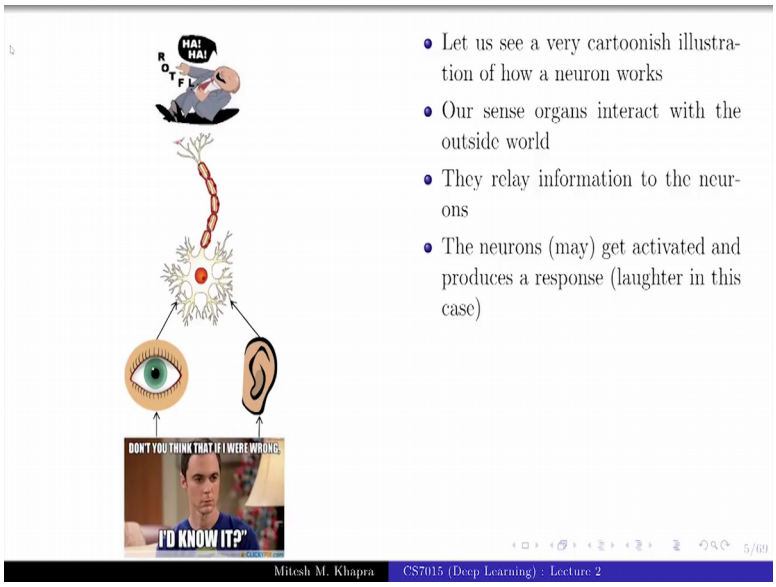
So, this is what a typical biological neuron looks like. So, here actually there are 2 neurons. This portion is called the dendrite, so it is used to receive inputs from all the other neurons.

So, that is the place where the input comes in. Then remember we said that in 1950s we discovered that these neurons are actually discrete cells and there is something which connects them. So, that connection is called a Synapse and it decides the strength of the connection between these two neurons. So, there is an input, there is some strength to the connection.

Then once this neuron receives inputs from various other neurons, it starts processing it, so that is the central processing unit which is called the SOMA, and once it is done this processing it will, it is ready to send its output to other set of neurons. So, that output is

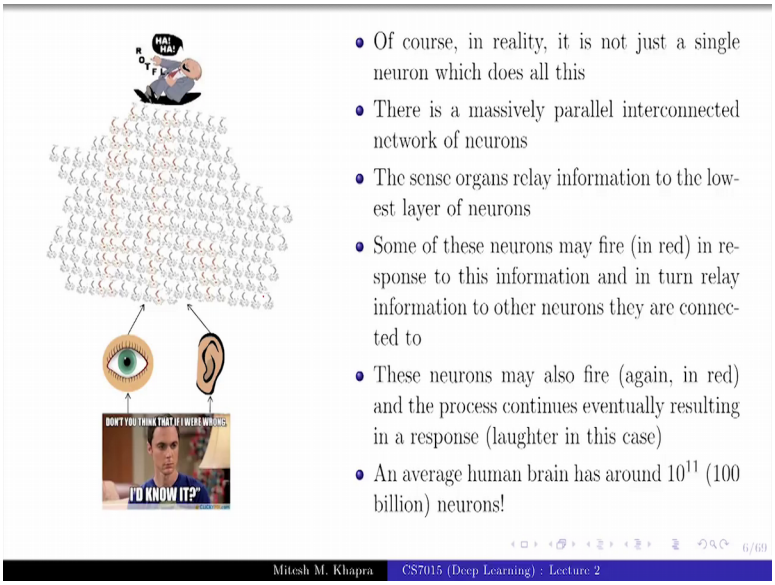
carried on by the axon. So, we have inputs, we have some weights attached to the input, we have some processing and then an output. So, that is what a typical biological neuron looks like.

(Refer Slide Time: 02:31)



And let us see a very cartoonish illustration of how this works right, how the neuron works. So, our sense organs interact with the outside world and then they pass on this information to the neuron and then the neuron decides whether I need to take some action. In this case the action could be whether I it should laugh or not right, whether the input is really funny enough to evoke laughter. So, if that happens this is known as something that the neuron has fired.

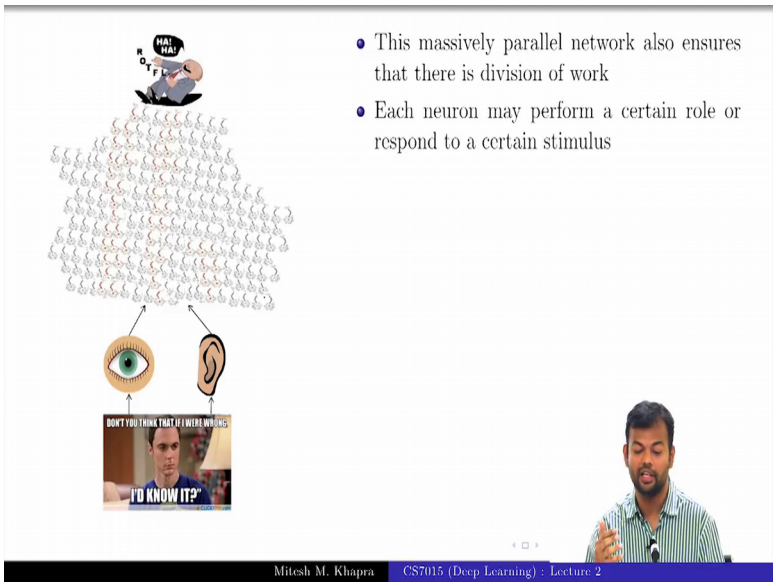
(Refer Slide Time: 02:57)



Now, of course, in reality it is not just a single neuron which does all this. There is a massively parallel interconnected network of neurons. So, you see a massive network here. Now the neurons in the lower level site, so these neurons. They actually interact with the sensory organs, they do some processing based on the inputs, so they decide whether I should fire or not.

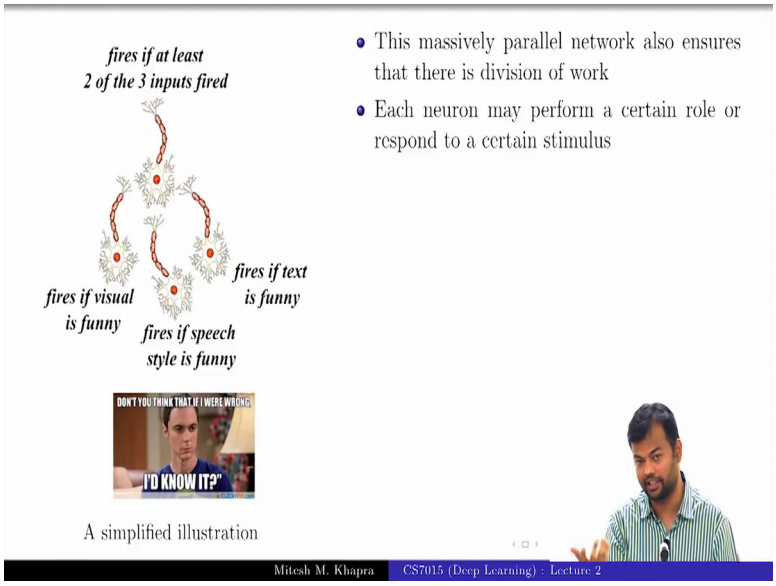
And if they fire they transmit this information to the next set of neurons and this process continues till the information is relayed all the way back and then finally, you decide whether you need to take any action or not, again in which this case it should be laughter, so that is how it works. And when I say massively parallel interconnected network I really mean it, because there are 10 raise to eleven which is roughly 100 billion neurons in the brain.

(Refer Slide Time: 03:44)



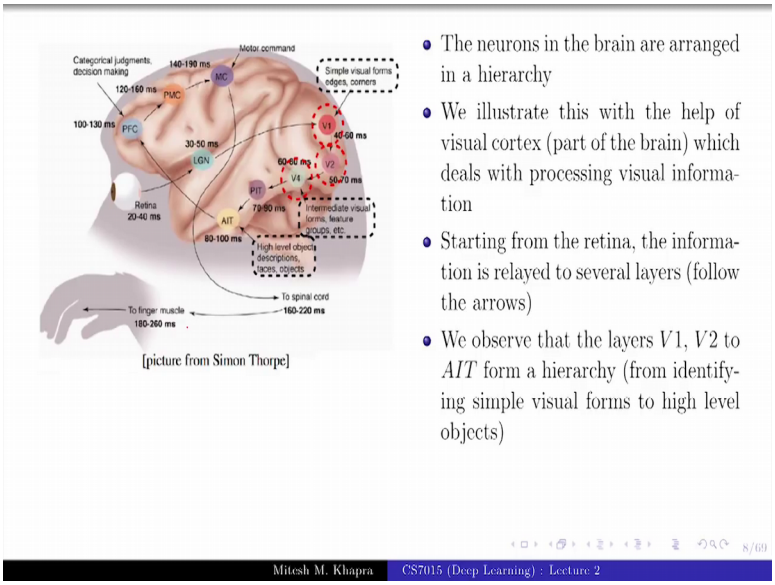
Now, this massively parallel network also ensures that there is some division of work. Now what do you mean by that is not that every neuron is responsible for taking care of whether I should laugh or not or not every neuron is responsible for processing visual data, some neurons may possess visual data, some neurons may possess speeds data and so on. So, there is this division of work, every neuron has a certain role to play. So for example, in this cartoonish example that we took.

(Refer Slide Time: 04:07)



So, there might be this one neuron which fires if the visuals are funny right whatever you are seeing is funny. There will be one neuron which finds Sheldons speech to be funny, the way he speaks, so that might be funny and there might be another neuron which actually finds the dialogue content to be funny. And now all of this pass on the information to the next level and this guy would fire if at least 2 of these 3 inputs are funny. So, that means, I have some threshold based on which I decide whether to react or not, if it is really funny then only I laugh it; otherwise I will not laugh.

(Refer Slide Time: 04:42)



So, the neurons in the brain as was obvious in the previous slide are arranged in a hierarchy and I will take a more realistic example, where we look at the visual cortex. So, is this is the portion of the brain, which is responsible for processing visual information right. So, as you see here you have our retina from where the information starts flowing, and it goes through various levels.

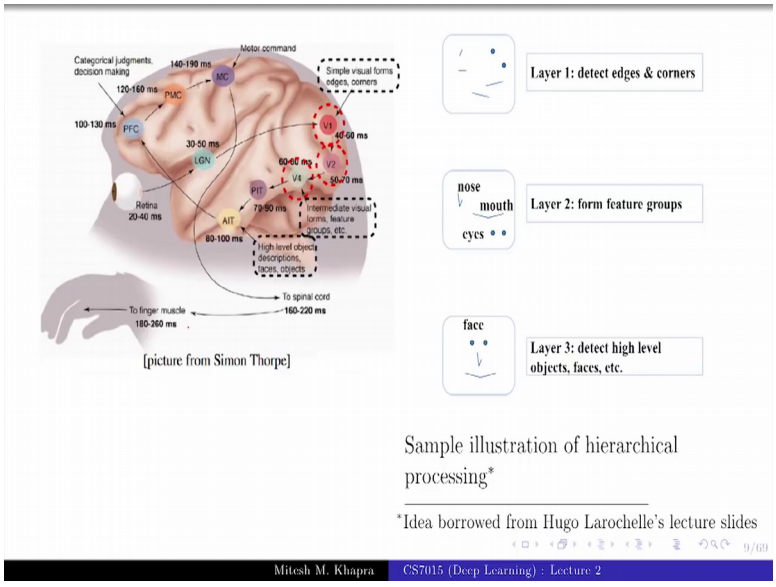
So, you see, you follow the arrows and you will see there are several levels; there is one level here, then another here another here and so on right. So, it is again as I was trying to illustrate in that cartoon the information is relayed through multiple layers, and then it goes all the way back to the spinal cord which decides that, in this case I need to move the muscle right.

So, that is what is being decided here right. So, the information flows through a hierarchy of layers. And in this particular case I am going to focus on these three circled

layers which are V1, V2 and AIT right. So, these actually form a hierarchy and let us see what this hierarchy does right.

So, at layer 1 you detect edges and corners. So, I am looking at you all, I just see some dots and some shapes, so that is what layer 1 recognizes. I just recognize some edges and some dots and so on.

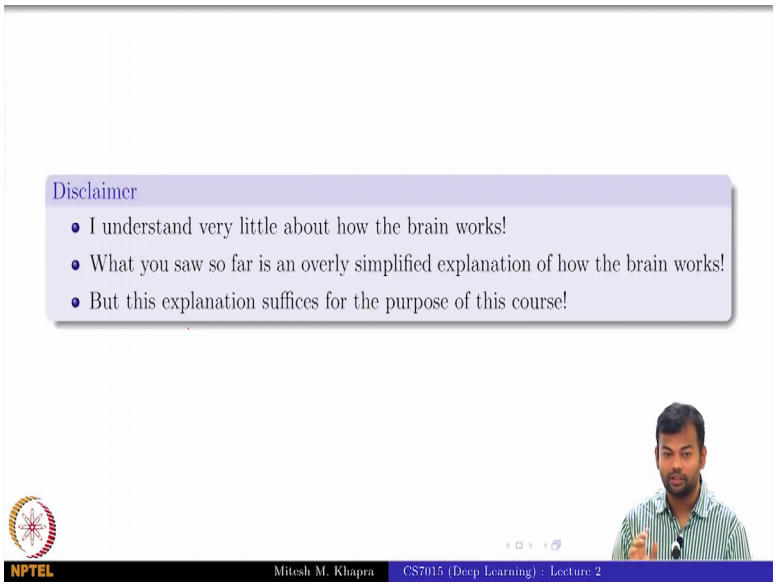
(Refer Slide Time: 05:40)



Now, layer 2 tries to group all of these together and come up with some meaningful feature groups right. So, it realizes oh these two edges actually form the nose, these two dots actually form the eyes and these two edges actually form the mouth right. So, that is slightly higher level of processing that it is doing and then layer 3 further collects all this and leads to higher level objects right.

So, now it is realizing all these things put together is actually a human face right. So, you add edges and circles or dots, then you had some feature groups and then the feature groups combine into objects right. So, that is how this hierarchy processes.

(Refer Slide Time: 06:30)



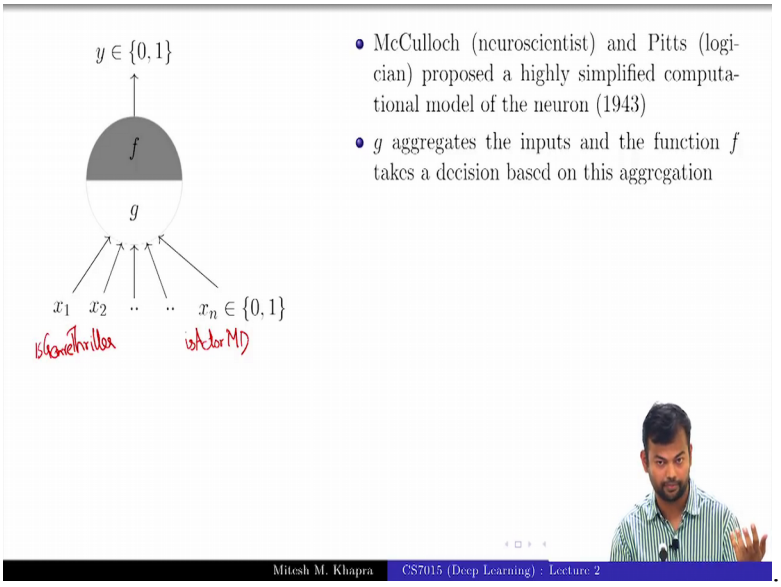
So, here is a disclaimer, I understand very little about how the human brain works right and what you saw is a very oversimplified explanation of how the brain works right. What I told you is there is an input a layer of networks which does a network, which has many layers which does some processing and then you have an output right; that is the very simplistic view that I gave you. This is an oversimplified version, but this version suffices for everything that we need for this course right. This is not a biology or a neural processing course right. So, it is enough for this course. So, that is where we will end module 1.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module 2.2
Lecture - 02
McCulloch Pitts Neuron

Let us start with module 2, which is about McCulloch Pitts Neuron.

(Refer Slide Time: 00:17)



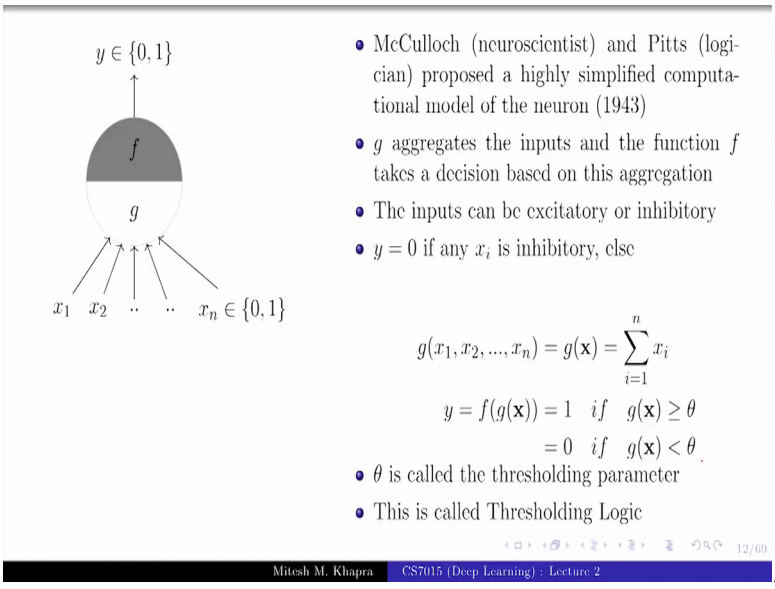
So, as we are done this during the history lecture way back in 1943, McCulloch and Pitts, they proposed highly simplified computational model of the brain. So, now let us see what's the motivation? We know that our brain is capable of very complex processing, it's capable of taking a lot of inputs from various sources and then, help us taking various decisions and actions. Now, what if you want a computer to do this. We want a module which is very similar to how the brain works or at least how we think the brain works which takes a lot of inputs and then does some processing and helps us take a decision.

So, what they proposed is this model which will take a lot of inputs and these inputs are all binary. All these inputs that you see here, these inputs are fed to this McCulloch Pitts neuron which is an artificial neuron and it is divided into two parts. So, the first part collects all the input. So, remember you had these dendrites which were taking all the information from everywhere. So, this just collects all the information and then the

second part is aggregation. I have collected a lot of information from all the sources. Now, the second function will decide what this aggregation is and based on that it will take a decision whether to fire or not.

So, the output is again Boolean. If it's 0, then neuron does not fire. If it's 1, the neuron fires. So, let us take a concrete example. So, suppose I am trying to make a decision whether I should watch a movie or not. So, x1 could be is the genre of the movie thriller. Similarly, there could be another variable say xn which says is the actor Matt Damon. So, these are all various such factors that I could take is the director Christopher Nolan, the music given by someone and so on. So, all these are probably factors which help me decide whether I want to watch this movie or not and you want this neuron to help us make that decision.

(Refer Slide Time: 02:21)



So, now what is happening here is these all inputs, they can be either excitatory or inhibitory. So, let me tell you what inhibitory is first. So, you are taking input from a lot of sources. Now, see one of these sources or one of these inputs is am I ill today? Am I down with fever? So, if that input is on irrespective of who the actor, director or whatever is, I am not going to watch the movie right because I just cannot leave from my bed. So, these are known as inhibitory inputs irrespective of what else is on in your input features. If this input is on, your output is always going to be 0. That means, the neuron is never going to fire. So, you could think of it as suppose my mood is not good today, I

do not feel like getting up or if I injured my leg or anything right, if any of these conditions is on irrespective of what the other factors are, I am not going to watch the movie.

So, that is an inhibitory input and excitatory input are on the other hand is not something which will cause the neuron to fire on its own, but it combine with all the other inputs that you have seen could cause the neuron to fire and how. So, this is how. So, these are all the inputs that your neuron is taking. All I am going to do is I am going to take a sum of these.

$$g(x1, x2, x3, \ldots., xn) = g(X) = \sum_{i=1}^{n} xi$$

I am going to take aggregation of all of these. So, what does this count actually give me the number of inputs which are on, the number of inputs which are value 1. That is all. This aggregate, this is a sum of all the one's in my input.

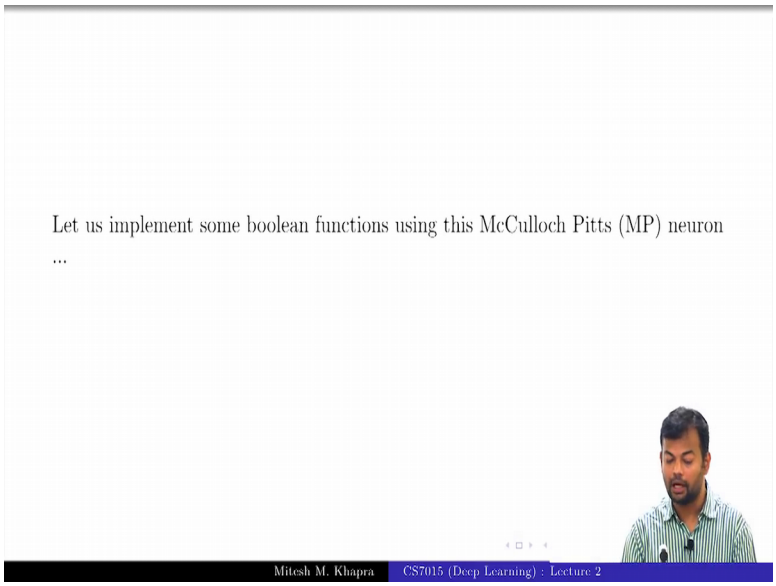
$$y = f\big(g(x)\big) = 1 \;\; if \;\; g(X) \geq \theta$$

$$y = f\big(g(x)\big) = 0 \;\; if \;\; g(X) < \theta$$

Now, this is what g does. This is a very simple function is taking a sum of my inputs. Now, the function y takes this as the input. That means, it takes this sum as the input and if the sum is greater than a certain threshold, then it fires. If the sum is less than the certain threshold, then it does not fire. So, again see what is happening here is it is same as now if you depend on the actor, director, and genre and so on and you fine. At least two of these three conditions are satisfied. At least I am happy with the actor and the director even though the genre is not something that I care about.

I will watch the movie or you might be a very niche go movie watcher who only goes to a movie if the actor matches your requirement, the director matches your requirement and the genre and the music and everything matches your requirement. So, you are threshold. In that case, it should be high. So, this is how it is going to help you make decisions. Now, again a very simplified model and this is theta is called the thresholding parameter that is the value which decides whether the neuron is going to fire or not and this over all thing is known as the thresholding logic. So, this is what a McCulloch Pitts neuron looks like.

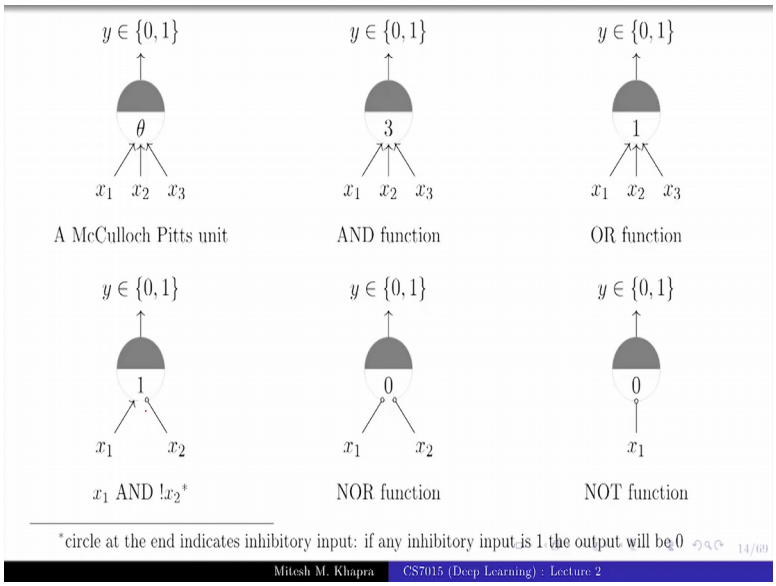
(Refer Slide Time: 04:52)



Now, let us implement some Boolean functions using this MP neuron. So, from now on I will just called it MP neuron and we will try to implement some Boolean functions using it. So, now why are we interested in Boolean functions? It is because we have overly simplified the way we take decisions. We are saying that the way we take decisions is we take a lot of Boolean inputs is actor Matt Damon and genre thriller and so on and based on that we produce a Boolean output.

So, an input is all Booleans. So, we have x1 to xn which are all Booleans and your output is also Boolean. So, that is a Boolean function that you are trying to learn from x to y. Is that clear? You have x just happens to contain n different variables here, ok and lot of decision problems you could cast in this framework. You can just imagine right whether to come for lecture today or not. Again is you could cast in it depending on various Boolean inputs.

(Refer Slide Time: 05:43)



This is a very concise representation of the McCulloch Pitts neuron. What it says is it takes a few Boolean inputs and it has certain threshold. If the sum of these inputs crosses this threshold, then the neuron will fire otherwise it will not fire. That is the simple representation of the m p neuron. Now, suppose I am trying to learn the AND function, when would the AND function fire?

All the inputs are on. So, what should be the value of the threshold in this case?

3. Everyone agrees. What about the OR function?

One. Let us see a few more this function. So, let me tell you what this function is. So, you see this circle here. So, that means that this input is an inhibitory input. If that is on, then the neuron is not going to fire. That is how I am representing it. So, now tell me what should the threshold for this be. It is not so hard.

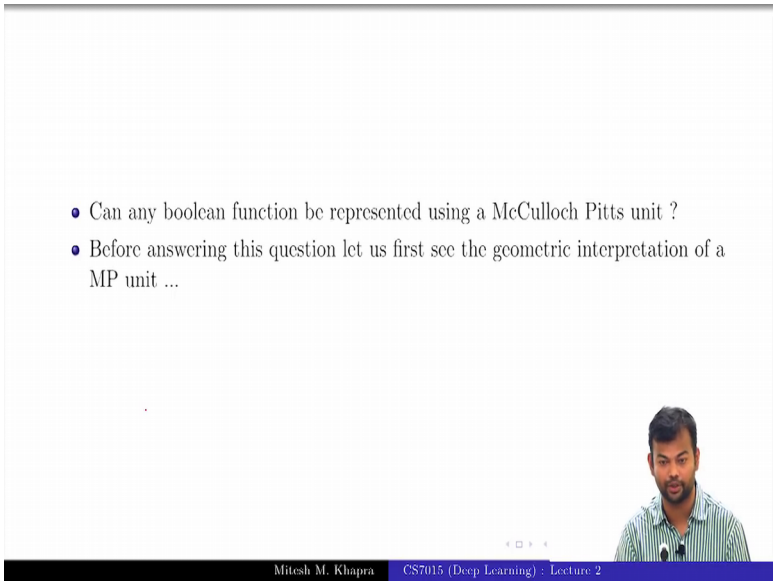
See if x2 is on, it is not going to fire. So, you have four rows; 0 0 0 1 1 0 1 1. So, two of those are ruled out and it is not going to fire. Now, out of the remaining two, when do you wanted to fire?

So, what should be the threshold?

One. Now, what about this function 0 or 3. 3 is not even a valid option. 0. Everyone agrees to that? and what about this? 0. So, you get this? So, now if you have a certain

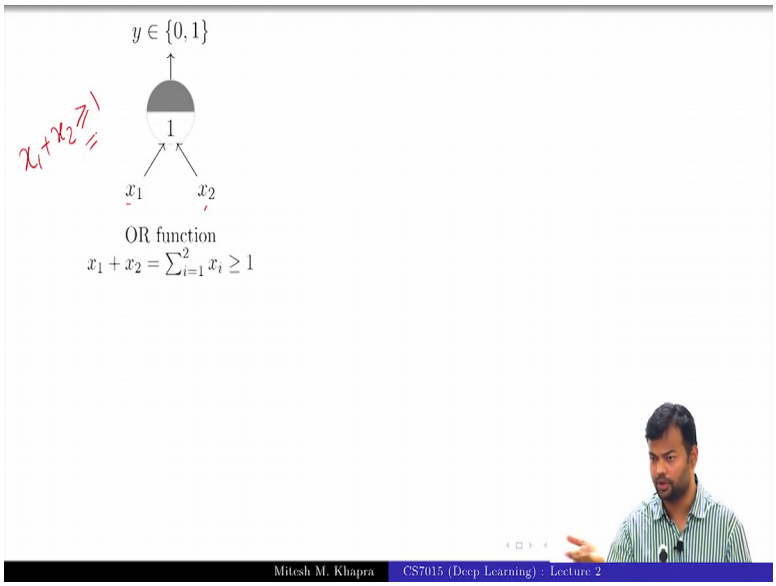
number of input variables and the function that you are trying to model the decision that you are trying to make is a Boolean function, then you could represent using these MP neurons whether all Boolean functions can be represented in this way or not that is still not clear. I am just showed you some good examples. We will come to the bad examples later on. Here is the question.

(Refer Slide Time: 07:37)



So, can any Boolean function be represented using a McCulloch Pitts neuron? So, before answering this question, we will see a bit of a geometric interpretation of what MP neuron is actually trying to do.

(Refer Slide Time: 07:49)



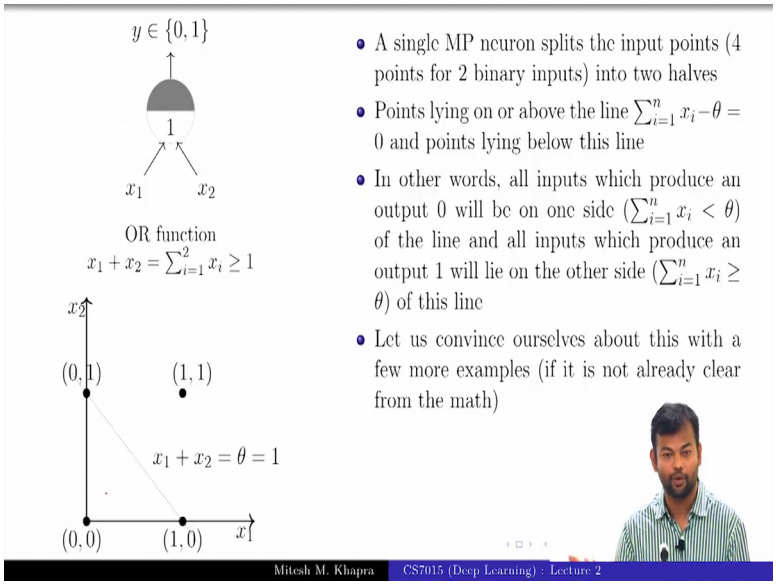
So, let us take OR function where you have two inputs x1 and x2 and this neuron is going to fire.

$$x1 + x2 = \sum_{i=1}^{2} xi \geq 1$$

If x1 plus x2 is greater than equal to 1. That is clear. That is how the definition is. Now, if you look at this, x1 plus x2 greater than equal to 1. Now, let us ignore the greater than part first. So, we will just talk about x1 plus x2 equal to 1. What is this equation of a?

Line everyone gets that. Ok, now, in this case since we are dealing with Boolean inputs and we have two access x1 and x2, how many input points can we have? 4, right 0 0 0 1 1 0 1 1.

(Refer Slide Time: 08:36)



So, you could have these four points. So, just note that this is an x1 and x2 axis, but only four inputs are valid here. This is not a real numbered access. This is only Boolean inputs possible here.

$$x1 + x2 = \theta = 1$$

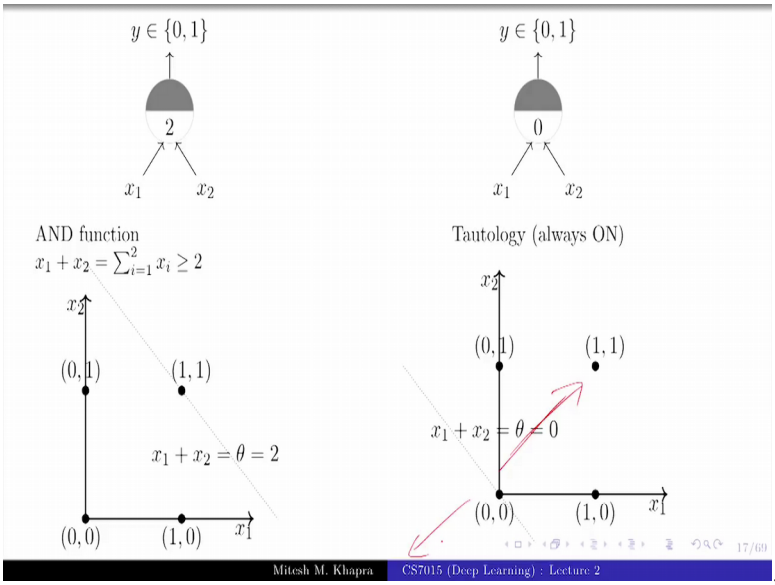
Now, what is the line x1 plus x2 equal to 1 tell you which line is that.

So, one which passes through 1, 0 here and 0, 1 here, this is that line. Now, what do we want that for all those inputs for which the output is actually 1, they should lie on the line or on the positive side on the line, and all those inputs for which the output is 0, they should lie on the other side of the line. Is that happening? So, what is actually MP in unit actually learning linear decision boundary. It just what it is doing in effect is actually it is dividing the input points into two halves such that all the points lying on that line right are, sorry all the points for which the input should be 0 lie below this line and all the points for which the output should be 1. Sorry in both cases, it should have been output.

So, let me just repeat it. All the points for which the output is 0 lie below this line and all the points for which the output is 1 either lie on this line or above the line. Is that fine? And so, let us convince ourselves about this. Even it is not already clear from the equation. For how many of you it is already cleared from the equation that this is exactly

what it does for a large number of periods, but still we will just do a few examples and move ahead.

(Refer Slide Time: 10:15)



Now, for the AND function what is the decision boundary? It is x1 plus x2. No, that is the decision boundary.

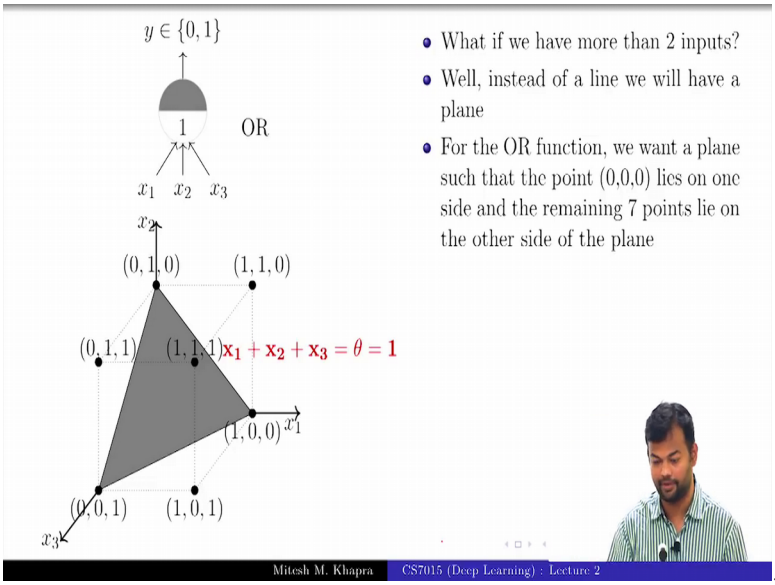
$$x1 + x2 = \sum_{i=1}^{2} xi \geq 2$$

Equal to 2, so, again I have these four points. Only these four points are possible. Now, where is my decision line?

Passing through that 1, 1 and intercepting this somewhere around 2, 0 and this around 0, 2. So, that is the line which I am interested in. Now, again do you see that our condition is satisfied that all the inputs for which we want the output to be 1 are on or above the line and all the inputs for which we want the output to be 0 or below the line. Now, what about this function? What is the threshold?

Zero, so, what would the line be? x1 plus x2 equal to 0 which passes through the origin, right and again all the points are either on or above the line. So, this part we are going to call as a positive half space and this we are going to call as the negative half space.

(Refer Slide Time: 11:24)



Now, what if we have more than two inputs? In a two dimensional case, when we just had x1 and x2 we are trying to find a separating line in the three dimension case. What will we do?

Plane. In the higher dimensions?

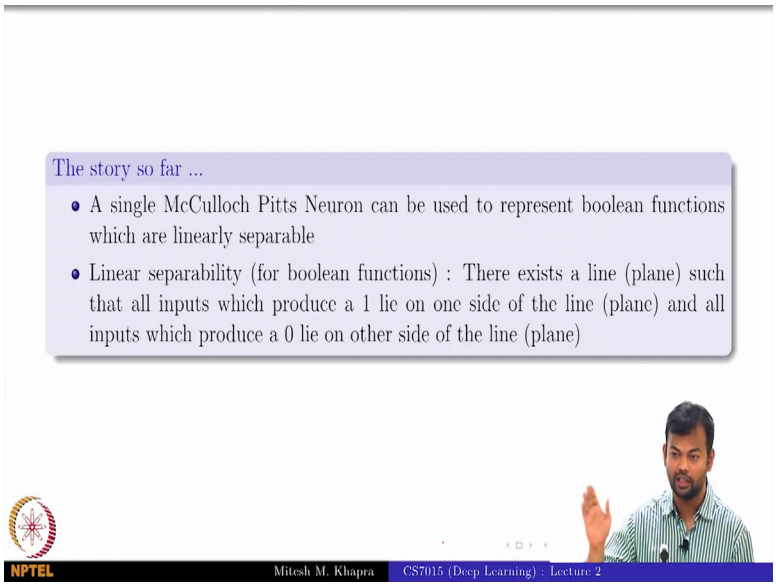
Hyper plane. So, this is now your three dimensional case. Again there are three axis here, but not all points are possible. How many points are possible? 8 points and which is the function that we are trying to implement.

OR. So, for these eight out of these eight points, for how many is the output 1?

7 and for 1, it is 0. So, what is the kind of plane that we are looking at? We are looking for a plane such that 7 points lie on or above it and 1 point lies below it and which is that point.

Zero, so now what is the equation of that hyper plane? X 1 plus x 2 plus x 3 is equal to 1. You see this. So, you see that all the seven points are visible, but the points 0, 0 is not visible because it is on the other side of the plane. So, this is doable in three dimensions also and again in higher dimensions also, right we could find in hyper plane.

(Refer Slide Time: 12:38)



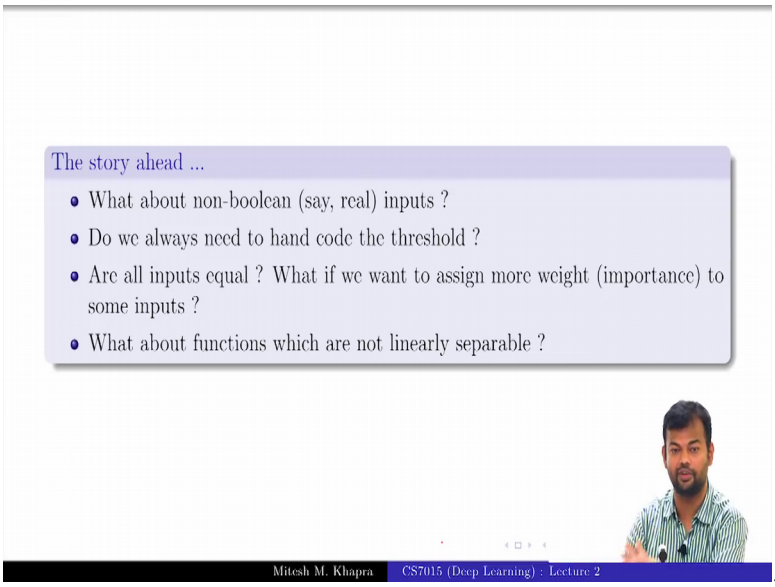
So, the story so far is that single McCulloch Pitts neuron can be used to represent Boolean functions which are linearly separable. So, a linearly separable function is such that there exists a line such that for that function whichever points produce an output of 1 lie on one side of the line and whichever points produce an output 0 lie on the other side of the line.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module 2.3
Lecture - 02
Perceptron

Now, let us go to the next module which is Perceptron.

(Refer Slide Time: 00:18)



So, far the story has been about Boolean input, but are all problems that we deal with, we are only dealing with? Do we always only deal with Boolean inputs? So, yeah so, what we spoke about is Boolean functions. Now, consider this example. This worked fine for a movie example where we had these as actor so much and his director and so on. But now consider the example where you are trying to decide. You are in oil mining company and you are trying to decide whether you should mine or drill at a particular station or not.

Now, this could depend on various factors like what is the pressure on the surface, on the ocean surface at that point, what is the salinity of the water at that point, what is the aquatic marina aquatic life at that point and so on. So, these are not really Boolean function. The salinity is a real number, density would be a real number, pressure would be a real number and so on, right and this is a very valid decision problem. Companies

would be interested in doing this. So, in such cases our inputs are going to be real, but so far McCulloch Pitts neuron only deals with boolean inputs. So, we still need to take care of that limitation.

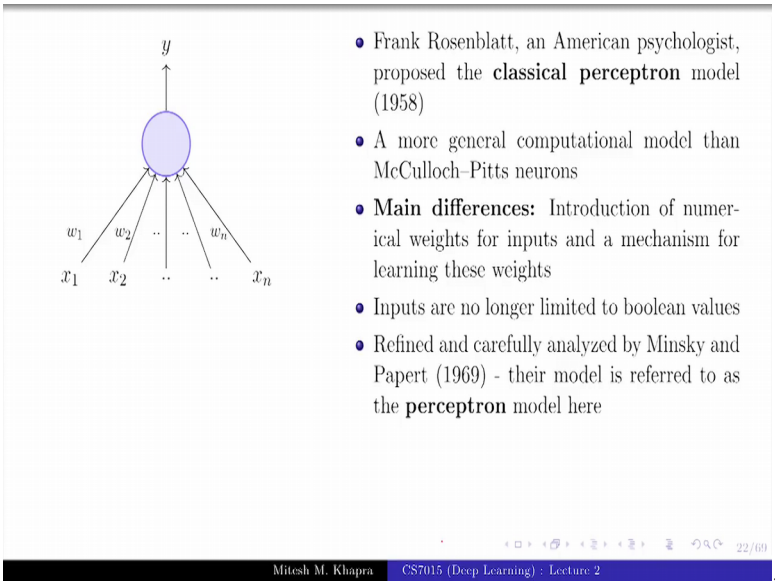
Now, how did we decide the threshold in all these cases? I just asked you, you computed it and you told me right, but that is not going to work out. I mean it does not scale to larger problems where you have many more dimensions and the inputs are not Boolean and so on. So, we need a way of learning this threshold.

Now, again returning to the movie example; maybe for me the actor is the only thing that matters and all the other inputs are not so important. Then, what do I need actually? I need some way of weighing these inputs. I should be able to say that this input is more important than the others. Now, I am treating all of them equal. I am just taking a simple sum.

If that sum causes a threshold, I am fine otherwise I am not fine, but maybe I want to raise the weight for some of these inputs or lower the weight for some of these inputs. So, whether it is raining outside or not maybe does not matter. I have a car, I could go or I could wear a jacket or an umbrella or something. So, that input is probably not so important,

And what about functions which are not linearly separable? We have just been dealing with the goody stuff which is all linearly separable, but we will see that even in the restricted Boolean case, there could be some functions which are not linearly separable and if that is the case, how do we deal with it. So, these are some questions that we need to answer.

(Refer Slide Time: 02:35)

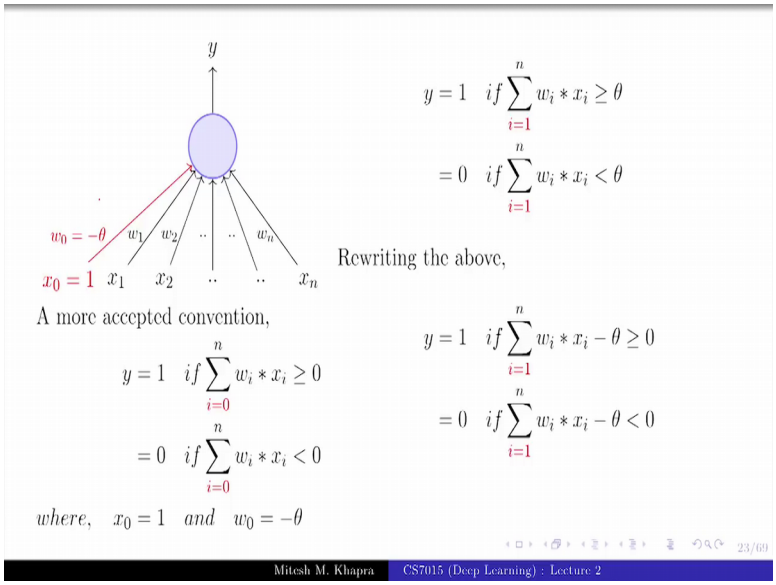


So, first we will start with perceptron which tries to fix some of these things and then, we will move forward from there. So, as we had discussed in the history lecture that this was proposed in 1958 by Frank Rosenblatt and this is what the perceptron looks like. Do you see any difference with the McCulloch Pitts neuron weights. You have a weight associated with each of the input otherwise everything seems.

So, this is a more general computational model than the McCulloch Pitts neuron. The other interesting thing is that of course we have introduced these weights and you also have a mechanism for learning these weights. So, remember in the earlier case, our only parameter was theta which we are kind of hand setting right, but now with the perceptron, we will have a learning algorithm which will not just help us learn theta, but also these weights for the inputs.

How do I know that actor is what matters or director is what matters? Given a lot of past viewing experience, past given a lot of data about the movies which I have watched in the past, how do I know which are the weights to assign this. So, we will see an algorithm which will help us do that, and the inputs are no longer limited to be Boolean values. They can be real values also. So, that is the classical perceptron, but what I am talking about here and the rest of the lecture is the refined version which was proposed by Minsky and Papert which is known as the perceptron model. So, when I say perceptron, I am referring to this model. So, this diagram also corresponds to that.

(Refer Slide Time: 04:06)



So, now let us see what the perceptron does. This is how it operates. It will give an output of 1 if the weighted sum of the inputs is greater than a threshold. So, remember that in the MP neuron we did not have these weights, but now we have these weighted sum of the inputs and the output is going to be 0 if this weighted sum is less than threshold. Not very different from the MP neuron.

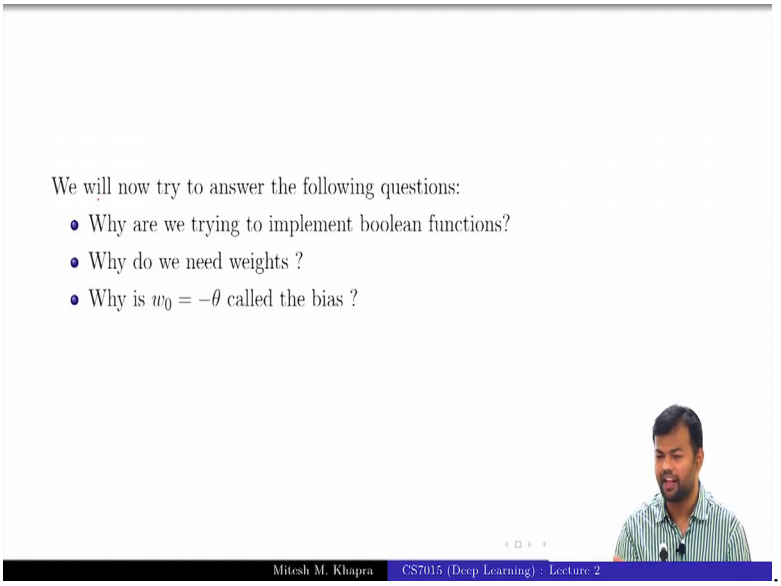
Now, I am just going to do some trickery and try to get it to a better notation or a better form. So, is this. I have just taken the theta on this side. Now is this. Notice this here the indices were 1 to n. Now, I have made it 0 to n and the theta is suddenly disappeared. So, what has happened.

Student: w 0 is.

Minus theta, right and x0 is 1. Does anyone not get this right. If I just start it from 1 to n, then it would be summation i equal to 1 to n wi xi plus w0 x0, but I am just saying w0 is equal to minus theta and x0 is equal to 1 which exactly gives me back this, right; So, very simple x0 equal to 1 and w0 is equal to minus theta. So, in effect what I am assuming is that instead of having this threshold as a separate quantity, I just think that that is one of my inputs which is always on and the weight of that input is minus theta. So, now the job of all these other inputs and their weights is to make sure that their sum is greater than this input which we have, does not make sense? So, this is how this is the

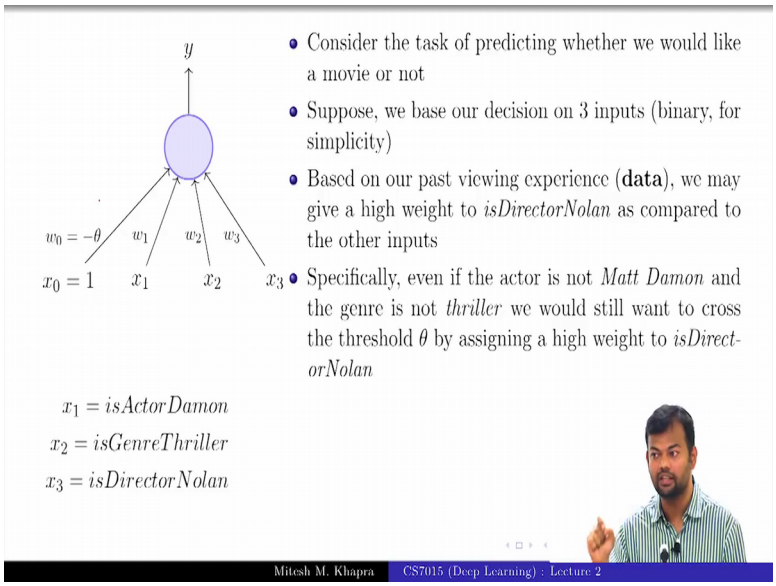
more accepted convention for writing the perceptron equation. So, it fires when this summation is greater than equal to 0, otherwise it does not fire.

(Refer Slide Time: 06:07)



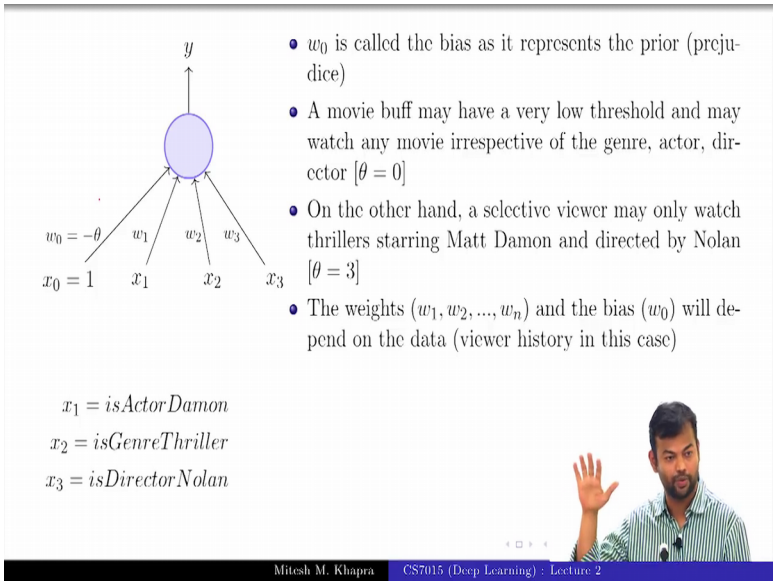
Now, let me ask a few questions. So, why are we trying to implement Boolean functions. I have already answered this, but I will keep repeating this question, so that it really gets drill in. Why do we need weights? Again we briefly touched upon that and why is w naught which is negative of theta often called the bias?

(Refer Slide Time: 06:25)



So, again let us return back to the task of predicting whether you would like to watch a movie or not and suppose we base our decisions on three simple inputs; actor, genre and director. Now, based on our past viewing experience, we may give a high weight to Nolan as compared to the other inputs. So, what does that mean? It means that as long as the director is Christopher Nolan, I am going to watch this movie irrespective of who the actor is or what the genre of the movie. So, that is exactly what we want and that is the reason why we want these weights.

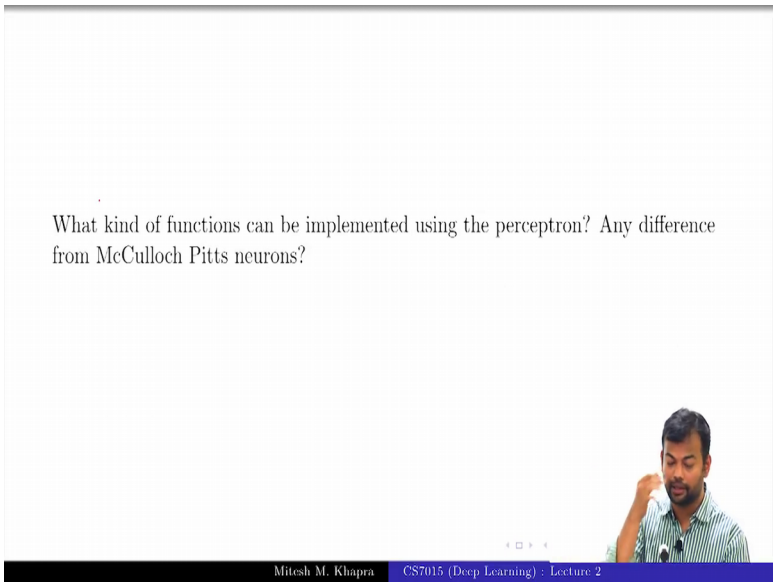
(Refer Slide Time: 06:58)



Now, w0 is often called the bias as it represents the prior. So, now let me ask a very simple question. Suppose you are a movie buff. What would theta be? zero. I mean you will watch any movie irrespective of who the actor, director and genre. Now, suppose you are a very niche movie watcher who only watches those movies which are which the genre is thriller, the director was Christopher Nolan and the actor was Damon, then what would your threshold be? 3.

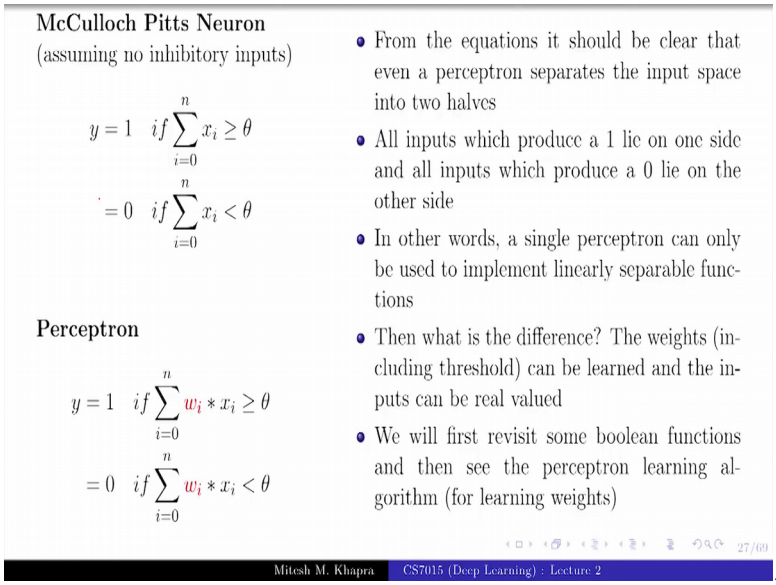
High in this case; I always ask this question do you know of any such movie always takes a while. Interstellar so, the weights and the bias will depend on the data which in this case is the viewer history. So, that is the whole setup. That is why you want these weights and that is why you want these biases and that is why we want to learn them,

(Refer Slide Time: 07:48)



Now, before we see whether or how we can learn these weights and biases, one question that we need to ask is what kind of functions can be implemented using the perceptron and are these function any different from the McCulloch Pitts neuron? So, before I go to the next slide, any guesses? I am hearing some interesting answers which are at least partly correct.

(Refer Slide Time: 08:11)

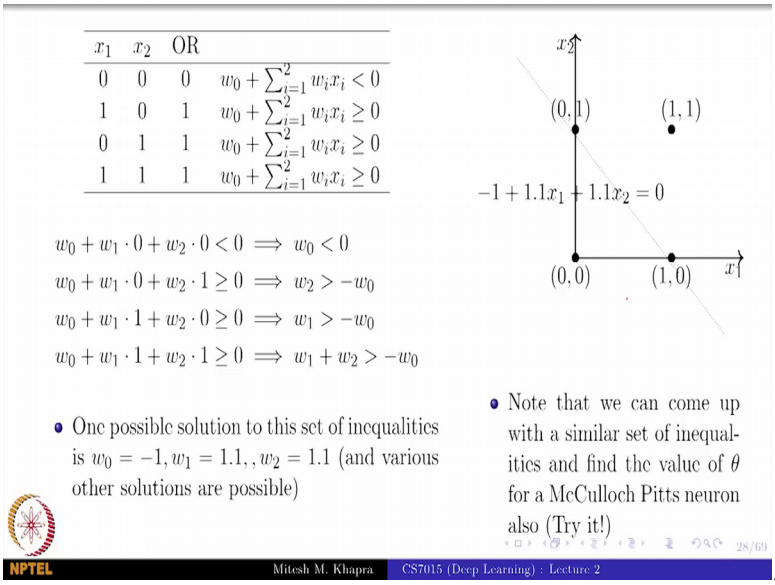


So, this is what a McCulloch Pitts neuron looks like and this is what a perceptron looks like. The only difference is this red part which is weights which has added. So, it is again

clear that what the perceptron also does is, it divides the input space into two halves where all the points for which the output has to be one, would lie on one side of this plane and all the points where which the output should be 0 would lie on the other side of this plane. So, it is not doing anything different from what the perceptron was doing. So, then what is the difference?

You have these weights and you have a mechanism for learning these weights as well as a threshold. We are not going to hand code them. So, we will first revisit some Boolean functions and then, see the perceptron learning algorithm.

(Refer Slide Time: 08:55)



So, now let us see what does the first condition. This condition if I actually expand it out, then this is what it turns out to be and what is that condition telling me actually w naught should be less than 0, clear. So, now based on these, what do you have here? Actually what is this? A system of linear inequalities, right and you know you could solve this. You have algorithms for solving this not always, but you could find some solution and one possible solution which I have given you here is w0 is equal to minus 1, w1 equal to 1.1 and w2 equal to 1.1.

So, just let us just draw that line. So, what is the line? It is 1.1 x1 plus 1.1 x2 is equal to 1. That is the line and this is the line and you see it satisfies the conditions that I have is this the only solution possible. No, right I could have this also as a valid line. If I could

draw properly, right all of these are valid solutions. So, which result in different w1 w naught and w 0s. So, all of these are possible solutions.

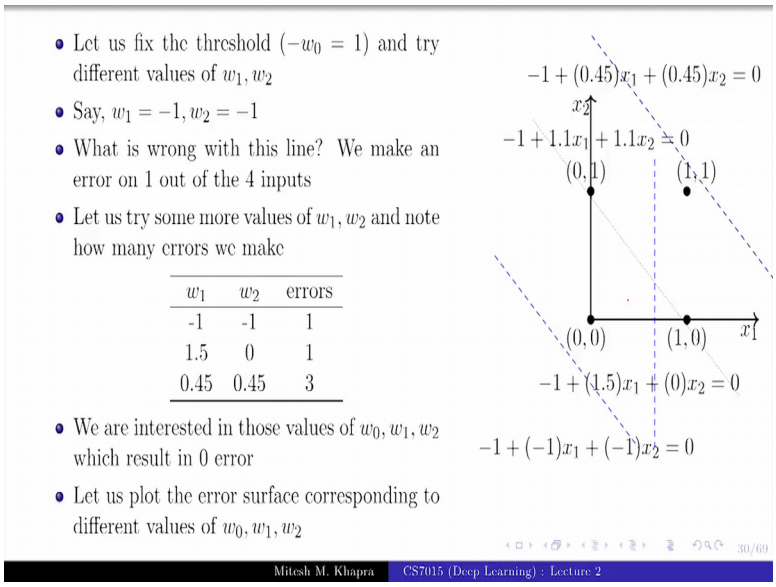
In fact, I have been telling you that you had to set the threshold by hand for the McCulloch Pitts neuron, but that is not true because you could have written similar equations there and then, decided what the value of theta should be. So, you could try this out for the McCulloch Pitts neuron also you will get a similar set of conditions or I mean similar set of inequalities and you can just say what is the value of theta, that you could set to solve that.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module 2.4
Lecture - 02
Errors and Error Surfaces

Before we go to the next section which is on learning, I just want to introduce the concept of Errors and Error Surfaces and tell you what it relates to these multiple solutions that we were talking about.

(Refer Slide Time: 00:26)



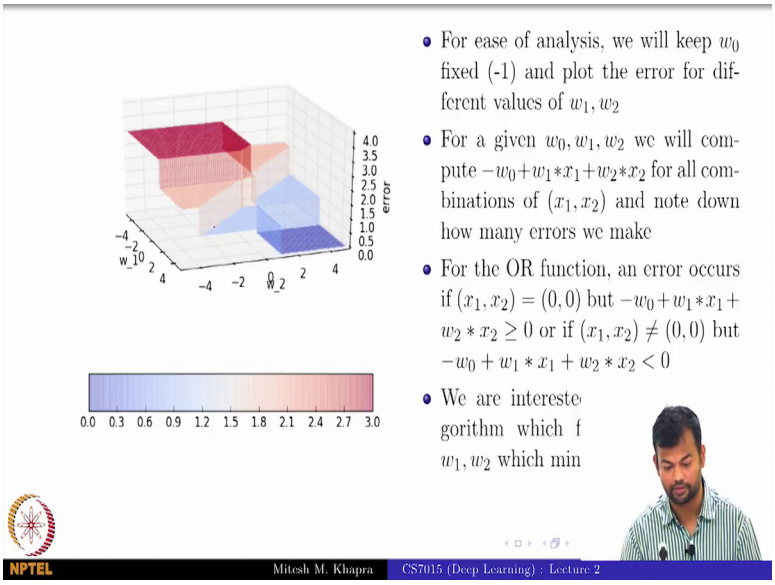
So, for simplicity what we will do is, we will just set the threshold to minus or minus w to 1 which is setting the threshold to minus 1 and now, I will try different values of w1 and w2 ok. So, I was saying that there are multiple values of w1 and w2 possible and these are all real numbers. We are not constrained by having them as Boolean values. So, now this is one solution which I tried. I tried setting w1 to minus 1 and w2 to minus 1. What is wrong with this line? Does it lead to any errors? How many?

Just one error. So, this makes an error of 1 out of the 4 inputs. Now, let me just try some other values of w1 and w2, this line; Again one error. What about this line? Not 4, 3 because 0, 0 is anyways on this side of a line.

So, now given this now tell me that I my quest is to find these w. So, I would want to find w1 w2 and so on. Given this discussion on errors, can you tell me a condition that I am looking for. I want to find w1 w2 or up to wn such that errors are minimized and in the best case errors are 0. So, that is what I want. So, this just I want to make a case that these search for w's is driven by certain objective and this objective is to minimize the error.

So, now since we are doing this, let us plot the error surface corresponding to different values of w naught, w1 and w2.

(Refer Slide Time: 02:03)



Once again for simpler analysis we will just keep w naught to be fixed at minus 1 and now what I have; so, just do not read this bullet as of now, even this one. So, I have this w2 here. So, that is my one axis and I have w1 here which is my another axis. Now, what I am going to do is I am going to try different values of w1 and w2. So, this axis can go from minus infinity to plus infinity. Of course, for showing the sake of showing, here I have just had it from minus 4 to 4.

So, now what I am going to do is I am searching for some values of w's w1 and w2. So, that my errors is 0 and let us do a brute force and I will just try every value between minus 4 to 4, ok. In fact, one of the solutions which I proposed actually was this 1.1, 1.1 right. That is the line which we saw on the previous slide and which led to 0 errors and that is the dark blue surface here. So, how did I compute this error? Actually I just

substituted minus, sorry 1.1, 1.1 here and then, I put in all the four values combinations for x1, x 2, and I realized that I am able to satisfy all of them. So, I do not get any error. Now, instead of that if I had put something different?

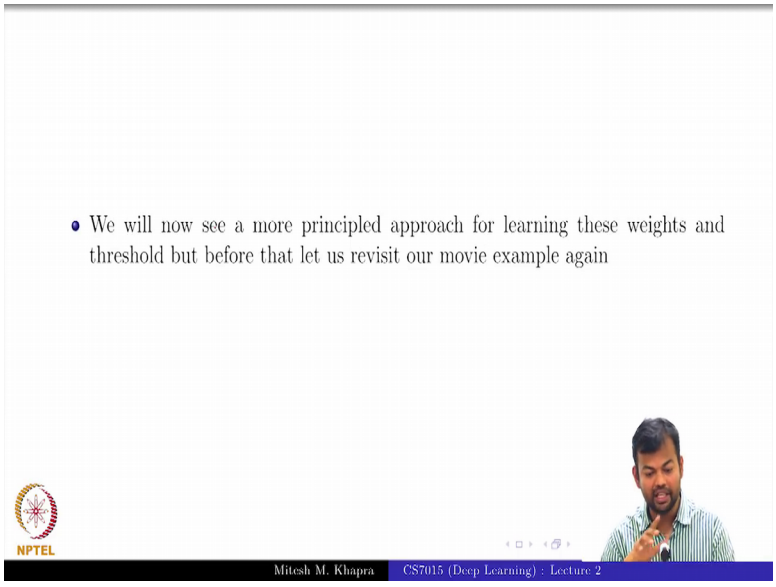
So, let me just go back to the previous slide which was see minus 1, minus 1 which is I think yeah somewhere around here right minus 1, minus 1 I guess. So, for that I am in this light blue region where the error was one. I make errors for one of the inputs. So, it is a very brute force way of finding this and this is not going to work, because we have lots of inputs to check, but this is just to give you an intuition that we are looking at errors and we are trying to find a value of w1, w2 which minimize this error. So, that is the idea behind errors and error surfaces.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 2. 5
Lecture - 02
Perceptron Learning Algorithm

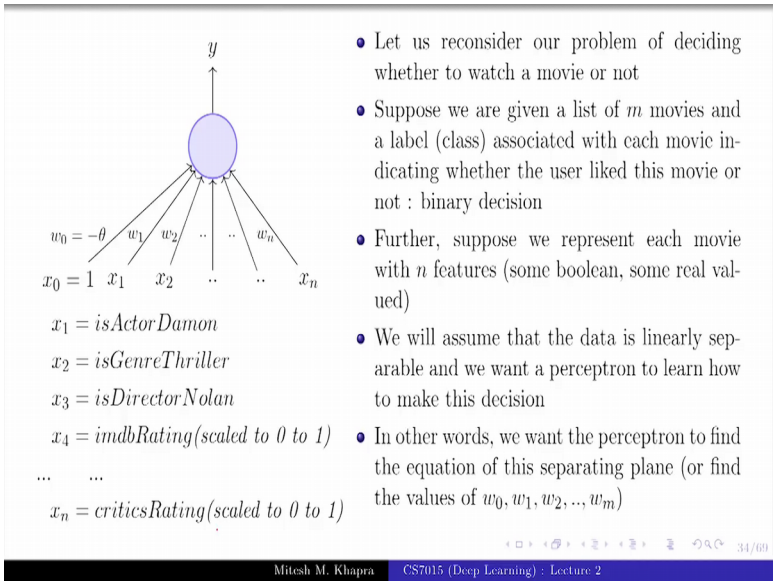
We will now go to the next module which is the Perceptron Learning Algorithm.

(Refer Slide Time: 00:17)



We now see a more principled approach of learning these weights and threshold, but before that we will just again revisit our movie example and make it slightly more complicated.

(Refer Slide Time: 00:25)

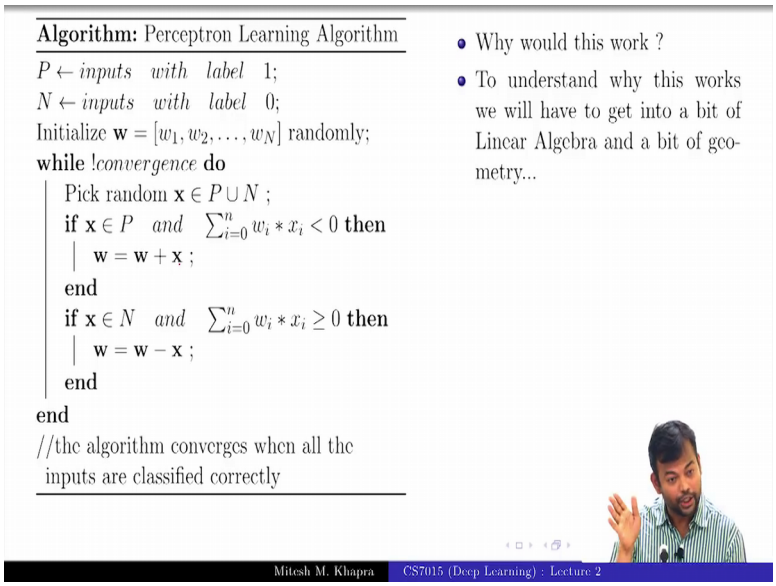


Now, here what the situation is that we are given a list of m movies and a class associated with each movie indicating whether we like the movie or not. So, now we have given some data of the past m movies that we have seen and whether we like this movie or not and now instead of these three variables, we have these n different variables based on which we are making decisions. And notice that some of these variables are real. They are not Boolean anymore. The rating could be any real number between 0 to 1, ok and now based on this data what do we want is the perceptron to do actually.

So, I have given you some data. These factors I have also given you the label 1 and 0. So, if the perceptron if I tell you my perceptron has now learnt properly, what would you expected it to do; perfect match. So, whenever I feed it, one of these movies it should give me the same label as was there in my data and again there are some movies for which I have a label 1 which are positive and some movies which I have a label 0.

So, I am once again looking to separate the positives from the negatives. So, it should adjust the weights in such a way that I should be able to separate. So, that is the learning problem that we are interested in.

(Refer Slide Time: 01:31)



So, now with that I will give you the algorithm. This is the Perceptron Learning Algorithm. We have certain positive inputs which had the label 1, we have certain negative inputs which had the label 0 and now I don't know what the weights are and I have no prior knowledge of what the weights are going to be. I need to learn them from the data. So, what I am going to do is, I am just going to initialize these weights randomly as I am also going to pick up some random values for this. So, this should be small n. So, this should be small n and now, here is the algorithm while not convergence do something.

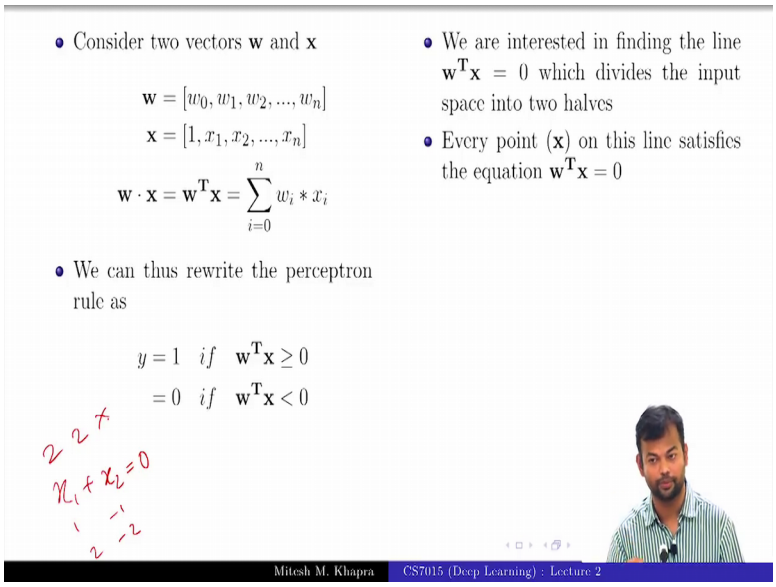
So, before I tell you what to do, can you tell me what is meant by convergence? When will you say that it has converged? When it is not making any more errors on the training data, right, or its predictions are not changing on the training data. So, that is the definition of convergence. Now, here is the algorithm. I pick up a random from point from my data which could either be positive or negative. So, it comes from the union of positive negative. Basically all the data that I have I pick up a random point from there.

If the point is positive, right and this is the condition which happens what does this tell me? If the point was positive, what did I actually want greater than 0, but the condition is less than 0. That means, I have made an error. So, I have made an error, then I will just add x to w I see a lot of thoughtful nodding and I hope you are understanding what is happening. Let us see. So, what is w actually? a dimensional.

N dimension n plus 1, right because w naught is also inside there. So, actually there should be w naught also here, right and what is x again n dimensional, right and that is why this addition is valid. So, let us understand that w and x both are n dimensional. Now, let us look at the other if condition. Can you guess what the other if condition is if x belongs to n and.

Summation is greater than equal to 0, then? So, that means you have completely understood how this algorithm works. Well, that is. So, now consider two vectors w and x. So, remember what we are trying to prove is or get an intuition. Not prove, actually get an intuition for why this works. ok

(Refer Slide Time: 03:55)



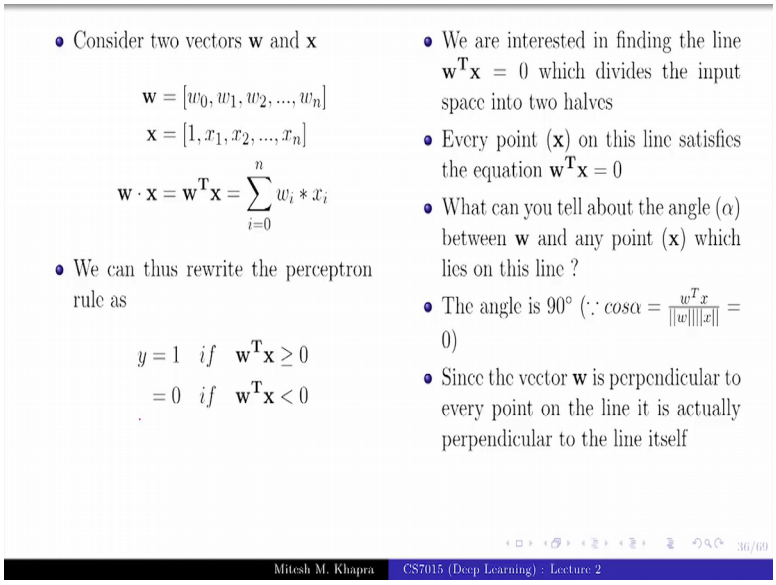
So, we will consider two vectors w and x and this is what my vectors look like very similar to the case that we are considering w0 to wn and 1 to n. So, this again x naught is just 1.

Now, this condition that I have been talking about is nothing, but the dot product. How many of you have gone through the prerequisites for today's lecture? Ok good. So, it is just a dot product. Now, we can just read write the perceptron rule as this instead of the dot product. I mean instead of using that summation thing, we can just say that it is a dot product.

Now, we are interested in finding the line w transpose x equal to 0. So, that is our decision boundary, which divides the input into two halves. Now, every point on this line satisfies the equation w transpose x equal to 0. What does that mean actually?

So, just a simple example is that if I have the line x1 plus x2 equal to 0, then all the points which lie on the line satisfy this equation. So, you could have 1 minus 1, 2 minus 2 and so on, but 2, 2 is cannot be a point on this line. At every point lying on this line satisfies this equation. So, every point lying on this line actually satisfies the equation w transpose x equal to 0.

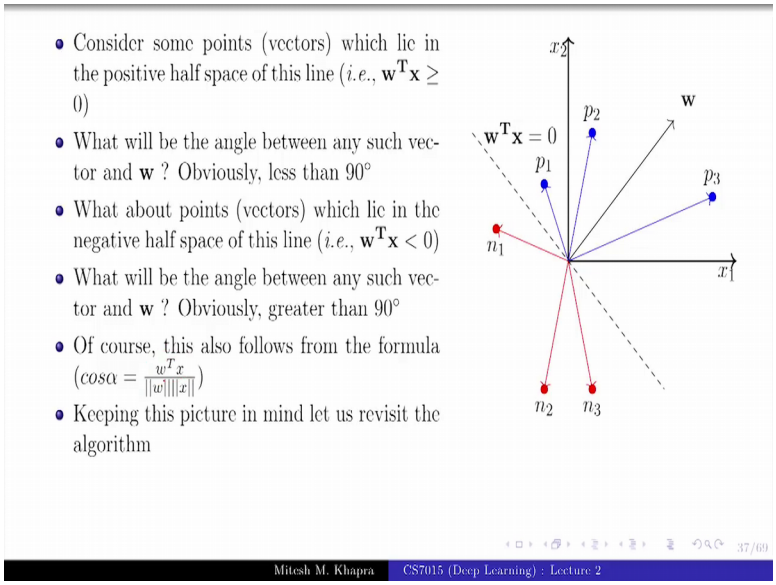
(Refer Slide Time: 05:14)



So, can you tell me what is the angle between w and any point on this line? How many say how many of you say perpendicular? Why?

Dot product is 0. So, if the dot product is 0, they are orthogonal. So, that means if I take this line, then my vector w is orthogonal to this. It is orthogonal to this point or this point to this point to every point on the line which is just the same as saying that the vector is perpendicular to the line itself, right as simple as that. So, the angle is 90 degrees because the dot product gives you the cos alpha and that is 0, right and since it is perpendicular as I said to every point of the line it is just perpendicular to the line itself.

(Refer Slide Time: 05:58)



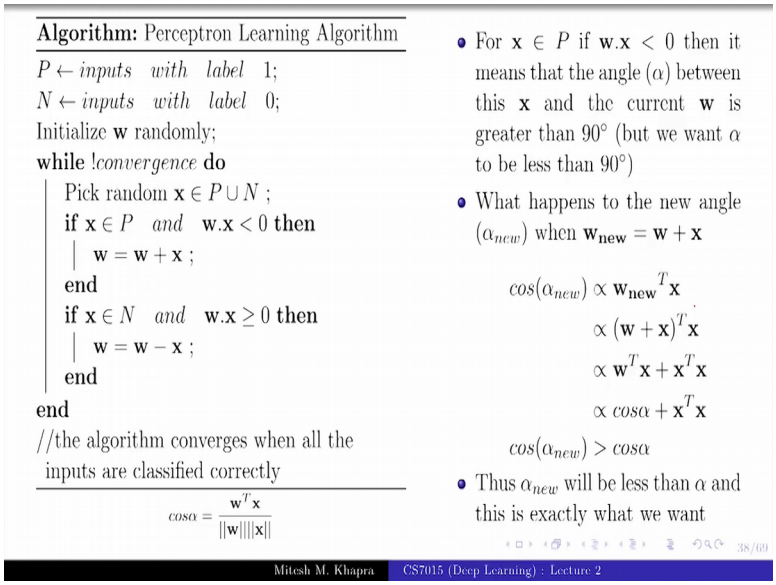
So, this is what the geometric interpretation looks like. This is our decision boundary w transpose x and the vector w is actually orthogonal to this line and that is exactly the intuition that we have built so far.

Now, let us consider some points which are supposed to lie in the positive half space of this line. That means these are the points for which the output is actually 1. Now, can you tell me what is the angle between any of these points and w or you guys are actually trying to tell me the angle we have got some measuring stuff, no. So, I will give you three options i. e. equal to 90, greater than 90 and less than 90.

Less than 90 it is obvious from the figure. Now, if I take any point which lies in the negative half space, what is the angle going to be between them? It is greater than 90. Again obvious and it also follows from the fact that cos alpha is w transpose x by something and we know that for the positive points w transpose x is greater than equal to 0. That means, cos alpha would be greater than equal to 0, that means the angle alpha would be less than 90 degrees and for the negative points w transpose x is actually less than 0. That means, cos alpha would be less than 0 that means alpha would be greater than 90 degrees.

So, it actually follows from the formula itself, but it is also clear from the figure. So, keeping this picture in mind let us revisit the algorithm. So, this is the algorithm.

(Refer Slide Time: 07:32)

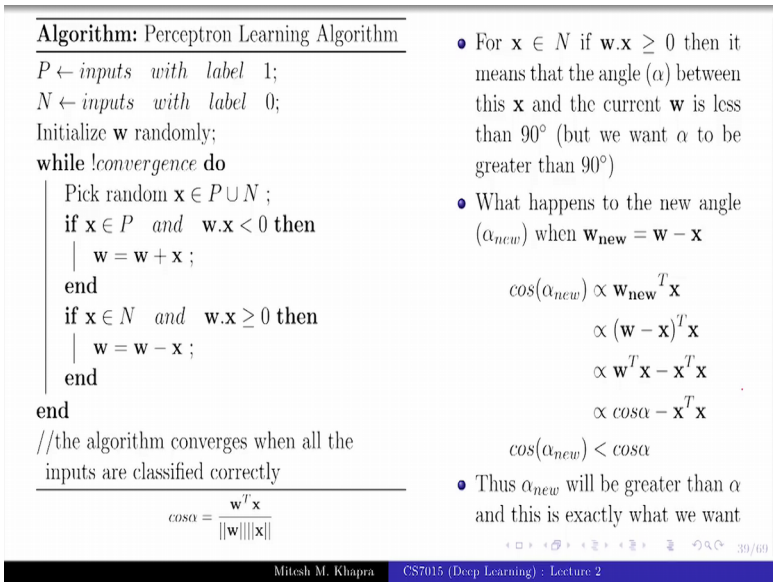


Now, let us look at the first condition which was this. Now, if x belongs to p and w transpose x is less than 0, then means that the angle between x and the current w is actually greater than 90 degrees, but what do we want it to be? Less than 90 degrees and our solution to do this is, but we still do not know why this works? Now, anyone knows why this works? So, let us see why this works.

So, what is the new cos alpha going to be? It is going to be proportional to this. It is going to be proportional to this. I will just substitute what w new is, fine. That means, if cos alpha new is going to be greater than cos alpha, what is alpha new going to be? It will be less than and that is exactly what we wanted. This angle was actually greater than 90 degrees. So, you want to slowly move it such that it becomes less than 90 degrees. It is not going to get solved in one iteration and that is why till convergence.

So, we will keep doing this. I will keep picking xs again and again till it reaches convergence. That means, till we are satisfied with that condition.

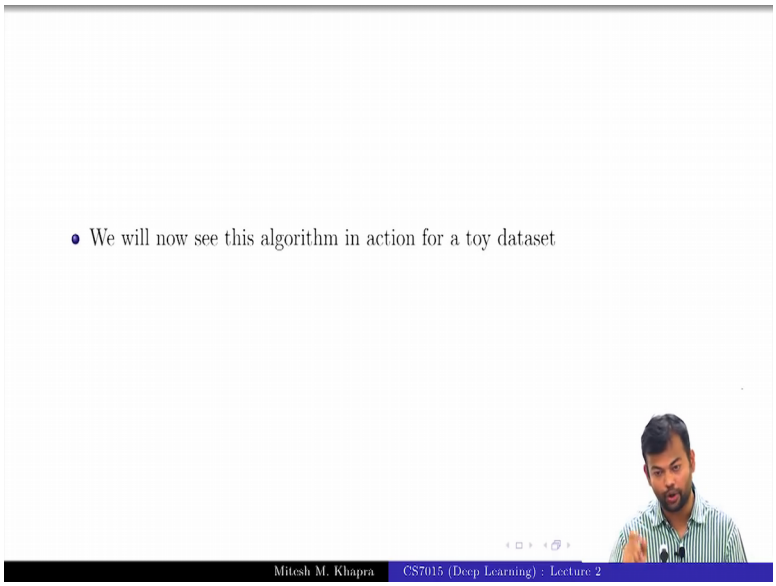
(Refer Slide Time: 08:47)



Let us look at the other condition x belongs to n and w transpose x was greater than equal to 0, then it means that the angle alpha is actually less than 90 degrees and we want it to be the opposite. I will just quickly skim over this w minus this x. Ok I forgot to mention that this is actually a positive quantity. I mean that is why that result holds. That means, cos alpha new is going to be less than cos alpha and this slight bit of mathematical in correctness, I am doing here, but that does not affect the final result.

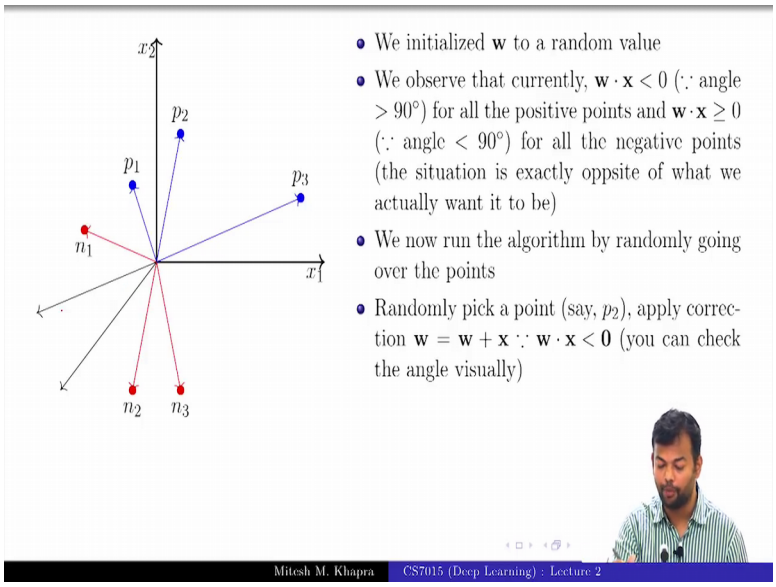
So, I will just gloss over that and you can go home and figure it out, but still it does not take away from the final intuition and interpretation. So, now the new cos alpha is going to be less than the original cos alpha; that means, the angle is going to be greater and that exactly what we wanted.

(Refer Slide Time: 09:40)



So, we will now see this algorithm in action for a toy data set.

(Refer Slide Time: 09:44)



So, this is the toy data set we have and we have initialized w to a random value and that turns out to be this. I just picked up some random value for w and ended up with this particular configuration for w.

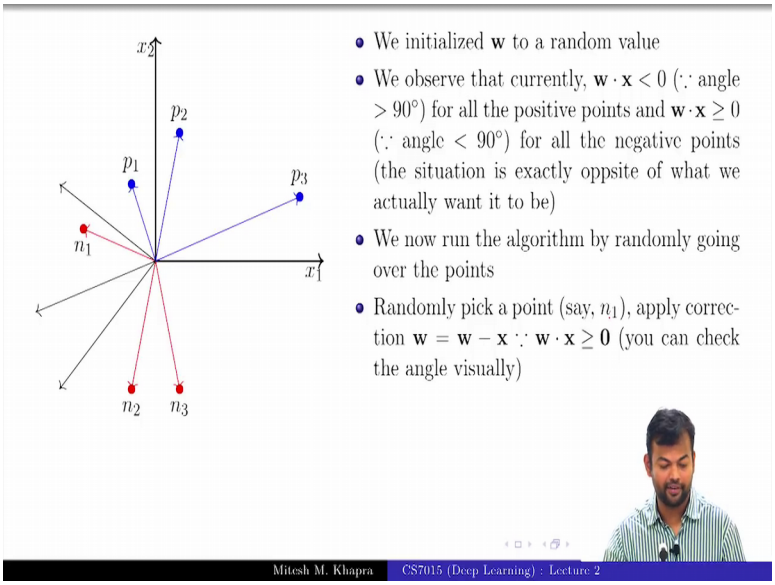
Now, we observe that currently w transpose x is less than 0 for all the positive points and it is actually greater than equal to 0 for all the negative points. If you do not understand w transpose x, it is just that the all the positive angle points actually have a greater than

90 degree angle and all the negative points actually have a less than 90 degree angle. So, this is exactly opposite of the situation that we want and now from here on, we want to actually run the perceptron algorithm, right and try to fix this w. How does it work? Remember we randomly pick a point. So, say we pick the point p1, do we need to apply a correction?

Yes. Why, because it is a positive point and the condition is violated. So, now we add w equal to w plus x and we get this new w. So, notice that we have a new w. We again repeat this, we again pick a new point and this time we have picked p2. Do we need a correction?

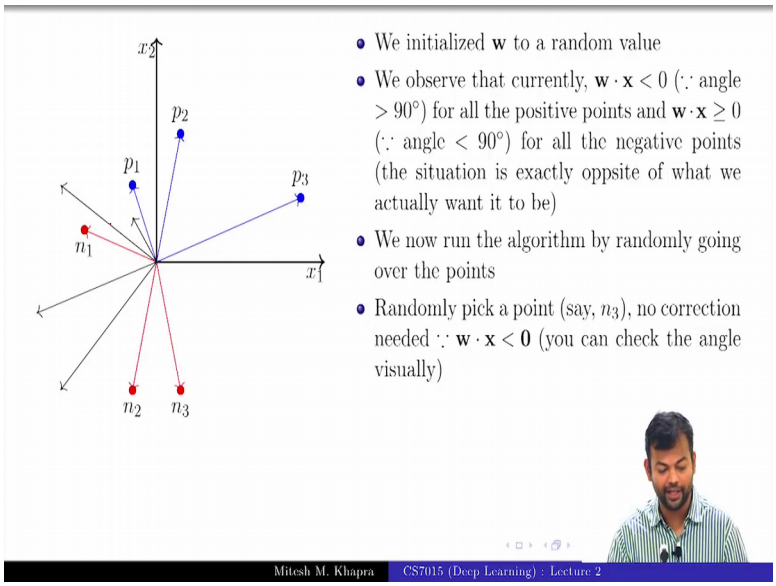
Yes, At least from the figure it looks like the angle is greater than 90. So, we will again do a correction. We will add w is equal to w plus p. This x is actually, sorry p2 and this is where we end up.

(Refer Slide Time: 11:12)



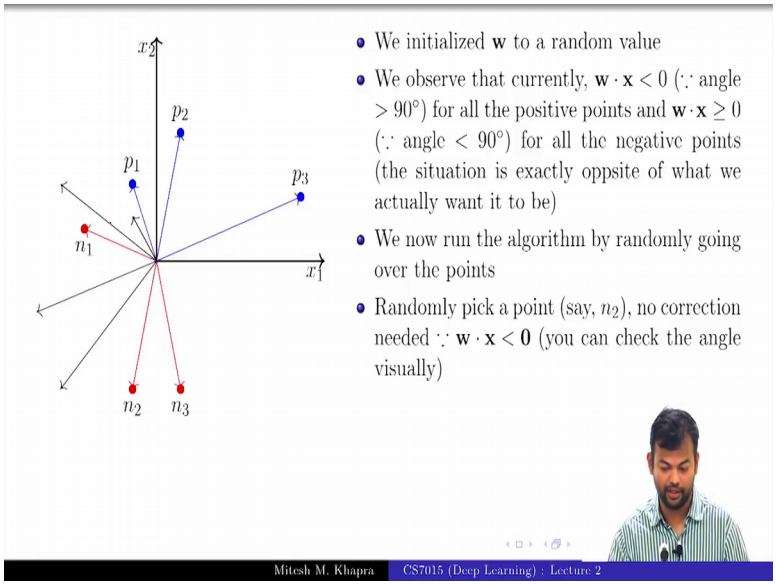
Now, again we pick a point randomly n1. Do we need a correction? So, this is what our w is. This line here and n1. So, we need a correction. Now, what is the correction going to be? It will be minus and then, the w changes.

(Refer Slide Time: 11:32)



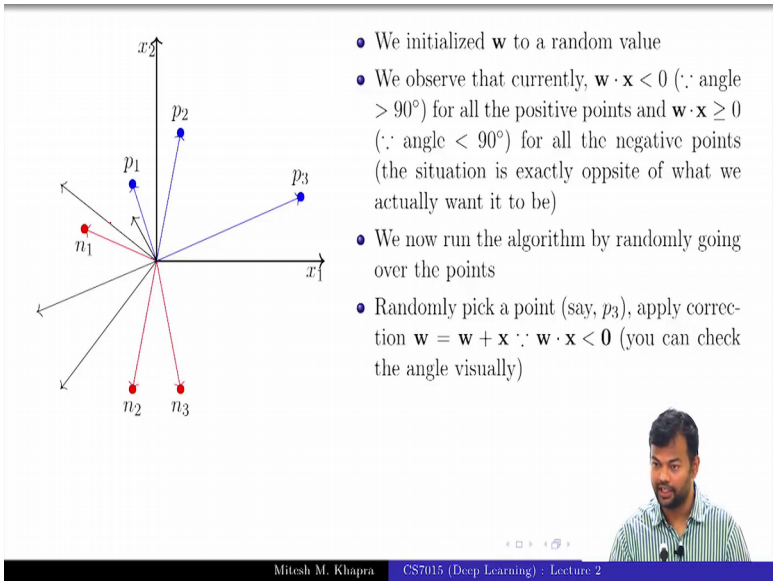
Now, we pick another point n3. Do we need a correction? No at least on the figure it seems like the angle is greater than 90 and we continue this.

(Refer Slide Time: 11:39)



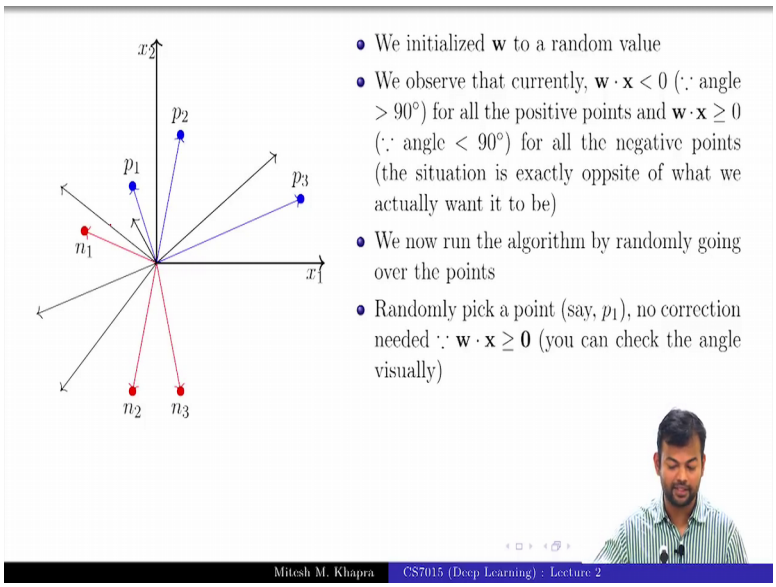
For n2, we do not need a correction. Now, for p3 again we do not need a correction.

(Refer Slide Time: 11:43)



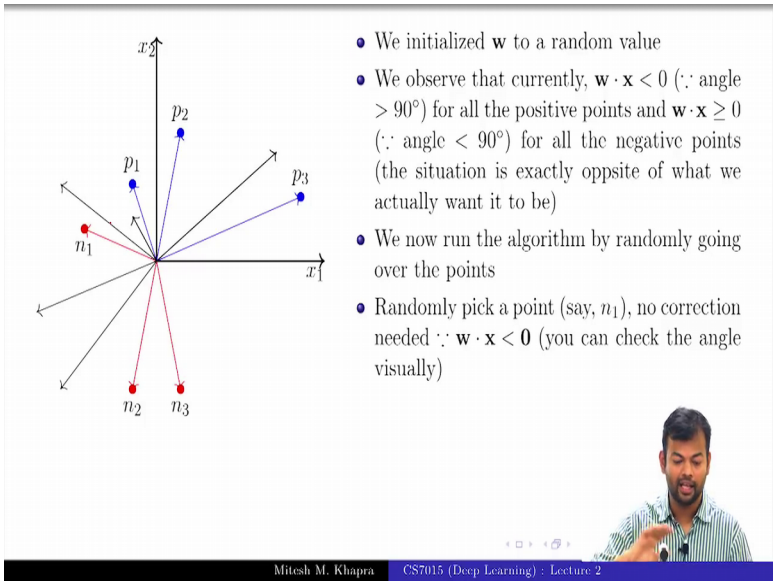
The angle looks less than 90. Sorry actually it is we need a correction. The angle is slightly greater than 90 and this is our correction, and now we keep cycling.

(Refer Slide Time: 11:52)



Now, as I keep cycling over the points, I realize that I no longer need any correction.

(Refer Slide Time: 11:54)



It should be obvious from the figure that for this particular value of w, now all my positive points are making an angle less than 90 and all my negative points are actually making an angle greater than 90. That means, by definition now my algorithm has converged. So, I can just stop it. So, I can just make one pass over the data. If nothing changes, I will just say it has converged. Now, does anyone see a problem with this?

It will never converge in some cases. So, can someone tell me why? We are considering only cases where the data is linearly separable. That we already assumed. So, what you are trying to tell me is that you are going over these points cyclically. So, let me just rephrase and put words in your mouth that what you are trying to tell me actually is that I take a point, I adjust w, but now for the next point I maybe go back to the same w because that point asked me to move it again and I keep doing this again and again and basically, end up nowhere. That is why this will never converge. That is exactly what you are trying to tell me.

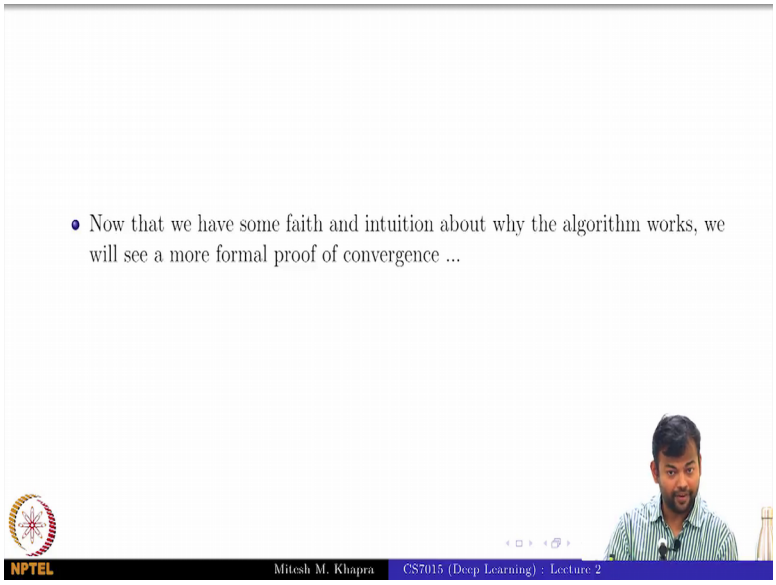
Now, that is exactly what I am forcing you to tell me. So, that is not the case. This algorithm will converge.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 2.6
Lecture – 02
Proof of Convergence

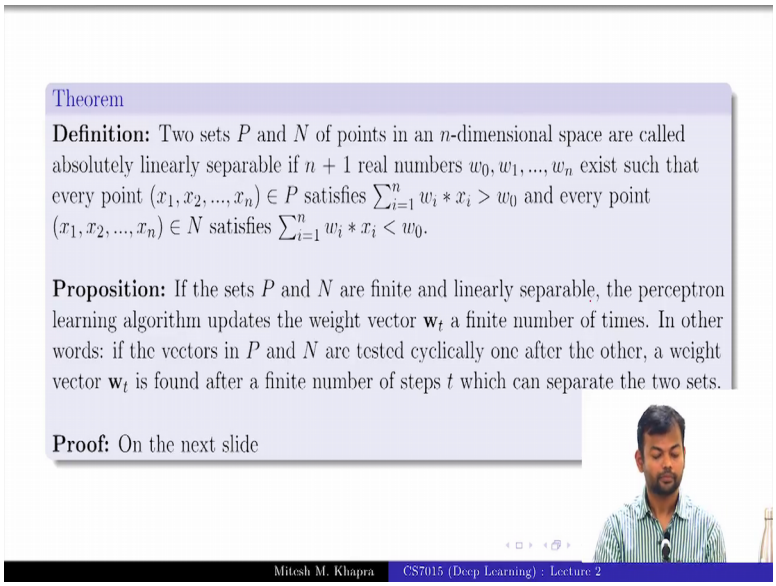
In this module we will talk about the Proof of Convergence for the Perceptron or the Learning Algorithm that we saw in the previous module.

(Refer Slide Time: 00:21)



So, we have some faith and intuition that it actually works, we just need to formally prove it that it actually converges. So, that is what we are going to do in this module.

(Refer Slide Time: 00:31)



So, before that a very few very simple definitions. So, if you have two sets of points P and N in an n dimensional space and we call say that these points are absolutely linearly separable, if there exists some n plus 1 real numbers which has w0 to wn; such that every point which belongs to P right, P is the case where the output is 1.

Then these set of weights satisfy this condition and every point which lies in the negative set the set of weights satisfy this condition. So, nothing very different from what has we have been saying so far, it is just formally defining it.

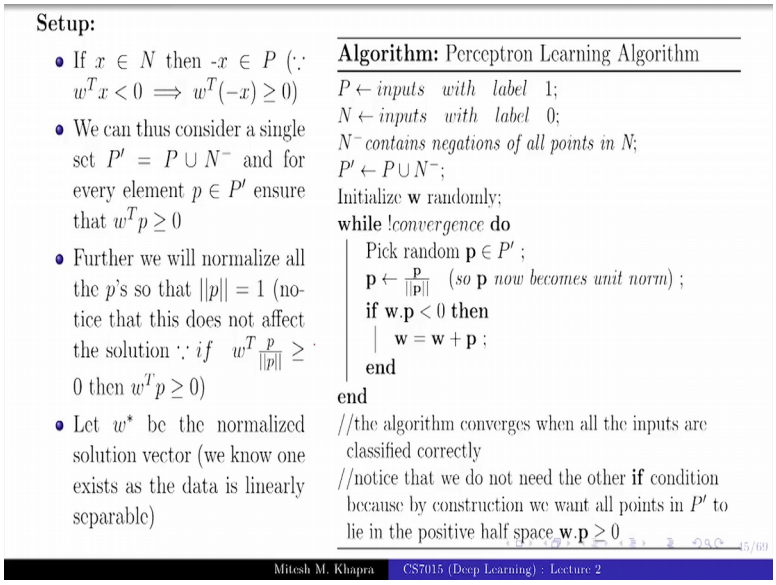
Now, our proposition is that, if the set P and N are finite and there is a fixed number of points in that which was the case in the toy example that we were doing and which will be the case in most examples that, we do and linearly separable. The perceptron learning algorithm updates the weight vector ok. Before I go there ok, let me not give you the definition and let me ask you the definition.

So, now, I have given this definition, the first definition and given this part of the proposition. Can you tell me what do I need to prove if I need to prove that the algorithm converges, that is one way of looking at it, but what was happening in that wrong argument which was I was making that it continuously kept toggling. That means, I am not making a finite number of updates right I have to keep changing again and again and this process continues in a loop.

So, that is how I am going to define convergence that the perceptron learning algorithm updates a weight vector of finite number of times, it only needs to update it finite number of times and it will reach a configuration such that now, it is able to separate the P from the N ok; that is what the proof of convergence means.

So, in other words if you are going to pick up these vectors randomly from the set P and N cyclically, as we were doing in the toy example, then a weight vector wt is found after a finite number of steps which will separate these two steps, these two sets. So, that is what we are trying to prove. So, that is the definition of converge, does it make sense?

(Refer Slide Time: 03:02)



So, proof is on the next slide and it is going to take me around 5 to 10 minutes to prove it. So, just stay focused all right. So, here is a few set up right. So, I am going to, before I go to the actual proof I am going to make a set up so that it becomes easier for us to prove it. So, the first thing that I am going to say is that, if there is a point which belongs in negative set then the negative of that point belongs in the positive set and that is very clear, because if the point belongs in the negative set then w transpose x is less than 0.

But then w transpose minus x would be greater than equal to 0 right? So, I take the negative of the point, I can just put it in the positive set. So, instead of considering these two different things P and N I am just going to consider one P prime, which is an union of P and all the N points negative ok, will the set up clear. If this is a setup then what is the condition that I need to ensure for every point in P dash.

Student: (Refer Time: 03:57).

W transpose p should be greater than equal to 0 right? So, I do not care about the negative case, I have just made everything positive now, and it is, I am not done anything wrong here, it is just a simple trick. ok And now, this is how the algorithm will look in this setup, these are the inputs with label one inputs with label 0, N minus contains a negation of all the points in N and P prime is a union of these. now, again I start by initializing w randomly, while convergence I will do something, I will pick a random P from P prime. now, what is the, if condition.

Less than 0.

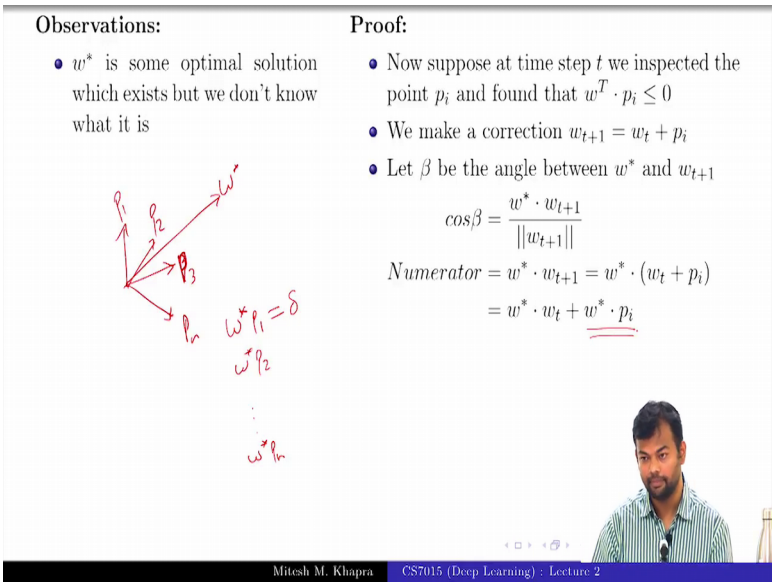
Do I need the other if condition?

No right, because everything is now, positive ok and the other small thing that I am going to do is, I am going to normalize P ok? So, that again does not mean, because we are talking in terms of angles and I am not changing the direction of the vector, I am just shrinking it right. So, I am just or maybe scaling it, also I am just making it unit norm. So, that does not change anything. So, it is still everything still holds.

And in particular you can see here. So, if this condition was true, this condition will also be true, ok. So, so far just I am done some simple tricks to make things easier for me later on, so now, P has been normalized. Now, remember that this data is linearly separable; that is what we started the proposition. If P and N are linearly separable then the perceptron learning algorithm will converge. So, now, if P and N are linearly separable, irrespective of whether we have the perceptron learning algorithm or not what do we know?

There exists a w star which is the solution vector right, there exists at least one w star which is the solution vector right; such that it will separate the P points from the N points. So, this vector which we do not know, but we just know, that it exists, so you can refer to it. So, we will call this w star fine? Now we start the proof.

(Refer Slide Time: 05:56)



So, w star is some optimal solution which we know, exists.

But we do not know, what it is right. now, suppose you had a time step t. So, remember that this algorithm is going on while convergence. So, you have time step 1 2 3 you are picking up points. So, we are at a time step t, at which you pick up a random point pi and you find that the condition is actually violated. So, this should actually be less than 0, if I know, the condition is violated. So, now, what will you have to do?

w is equal to.

w1. So, I will just call it the new w wt plus 1 is equal to the old w plus pi, ok. now, what I am going to do is I am going to consider the angle beta between w star and wt plus 1. I do not know, what w star is, but we can still assume it exists and make some calculations based on that. So, what is the angle between w star and wt plus 1, its beta and what is the cost of that angle this

And remember that we do not have w star here, because we had assumed that it is the normalized vector. So, we do not need that but, this is actually equal to 1 ok So, now, if I just take the numerator, w star in dot product wt plus 1 now, I am going to expand wt as wt plus pi fair; that is exactly what I did on the previous step.

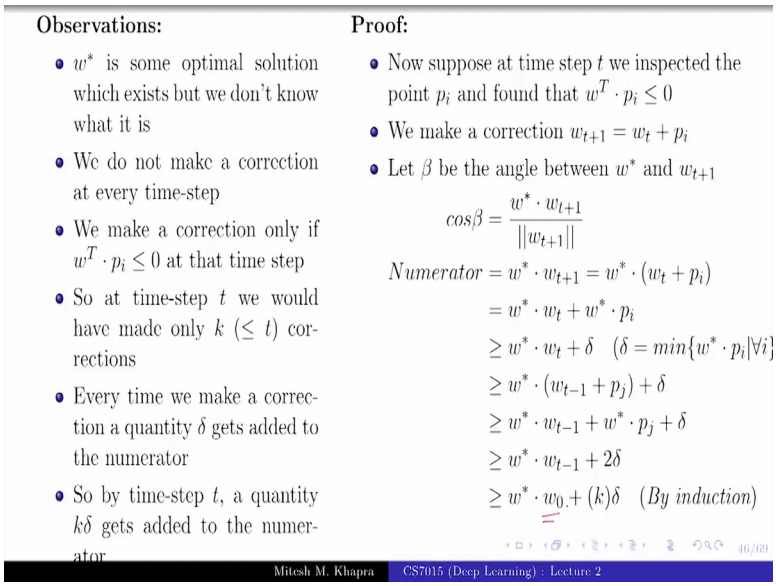
Now, now, what is pi actually, it is. So, what you had is you had these p1, p2, p3. My hand writing is really horrible and up to pn right. So, I have just picked one of these pi's.

ok? Now, what I am going to define is, now, suppose this is my, these are my pi's. So, these are all the vectors that I have. Now, suppose I have this w star, suppose this was the w star that I am interested.

Now, for each of these I could compute w star p1, w star p2 and so on up to w star pn and I could sort them. Now, what I am doing is that for whichever of these points w star pi is the minimum, I am going to call that value as delta. Suppose w star P 1 is the smallest quantity out of w star p1 w star, p2 w star, pn, and I am calling that quantity delta.

So, I have this quantity here and my delta is the minimum of all the possible values that it can take. It can make w star p1 p2 up to pn, so delta is the minimum quantity. So, here I have an equality.

(Refer Slide Time: 08:45)



Now, are you ok with this? This is the minimum quantity right. So, any pi that I put in here it is always going to be greater than or worst case equal to delta.

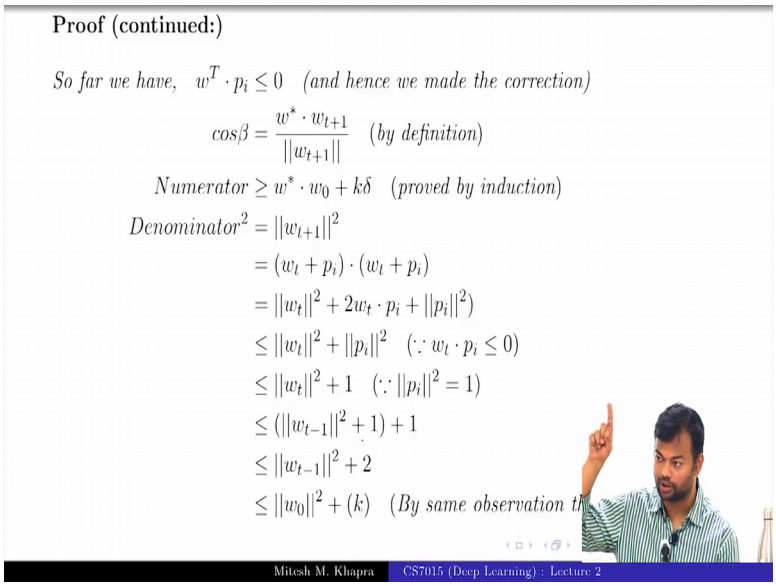
Now, again this w2 itself I could write it as wt minus 1 plus pj, because that also would have come up from some update in the previous step. ok Again this is there which I could call it as delta and still retain the greater than equal to here, ok fine. So, let us see where are we heading with this.

Now, notice that we do not make a correction at every time step, when I was running that toy algorithm I was not making a correction at every time step. We were only making a correction at those time steps for which the condition was violated. So, now, if I am at t'th time step, maybe I have made only k which is less than or equal to t corrections. At max I would have made t corrections, but it could have been less than that also.

So, now, every time we make a correction we are adding a value delta to this. So, at the time step t what would happen, I had started off from w naught I have reached time safety and I have made a case that, I have not made t updates I have made k less than equal to t updates.. So, how many deltas would get added?

K delta. So, I could say that with respect to w naught where I had started from, this is what this quantity is, ok is that fine, anyone has a problem with this.

(Refer Slide Time: 10:19)



So, far what are we shown, we started with this, this condition was true again not less than equal to and hence we made the correction and this was the point that we picked up at the t th step and thence we made that correction.

And we also showed that the numerator is actually greater than equal to this quantity, we showed it by induction fine. Now, let us look at the denominator and particularly let us look at the denominator squared, ok is a step right.

This is actually wt plus 1 dot product wt plus 1, but wt plus 1 can be written as wt plus p i. This bracket needs to disappear right, is that ok fine. Now, what is, what is this quantity?

That is less than equal to 0. So, now, can you guess what is the next thing that I am going to write?

That is correct, yeah it is a negative quantity. So, that is going to be less than equal to this. So, that is fine and what about pi square or this term.

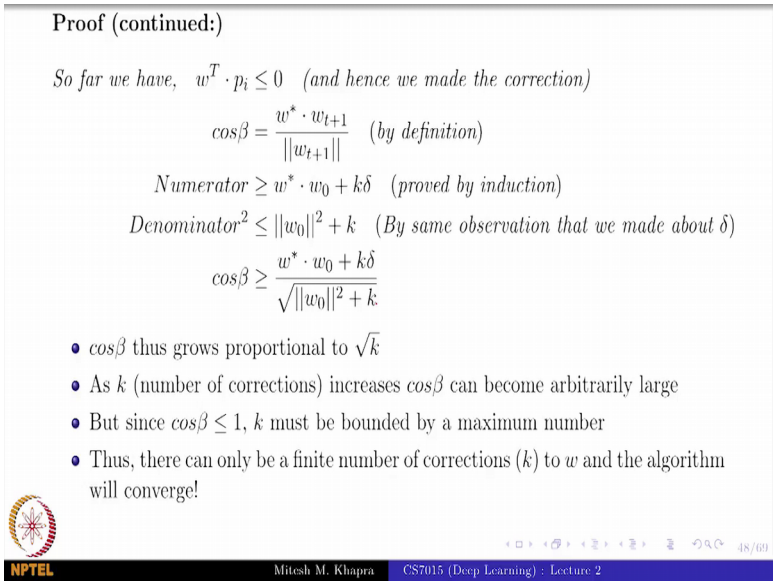
Because this is less than right that is why.

Correct is this fine? ok. Now, what is pi square?

One. Now, can you guess what I am going to do? by induction?

So, what is wt square again? Just this wt plus 1 square was wt square plus 1, wt square is going to be wt minus 1 square plus 1 right and how many such ones will get added? k of those right, starting from w naught ok

(Refer Slide Time: 12:36)



So, what have we shown, the numerator is greater than equal to this, the denominator is less than this. ok now, if I put them together I actually get that cos beta is going to be greater than equal to the numerator over the denominator. Ok now, what is this quantity proportional to k, k square, k cube square root of k, k by 2?

Student: Square root of k.

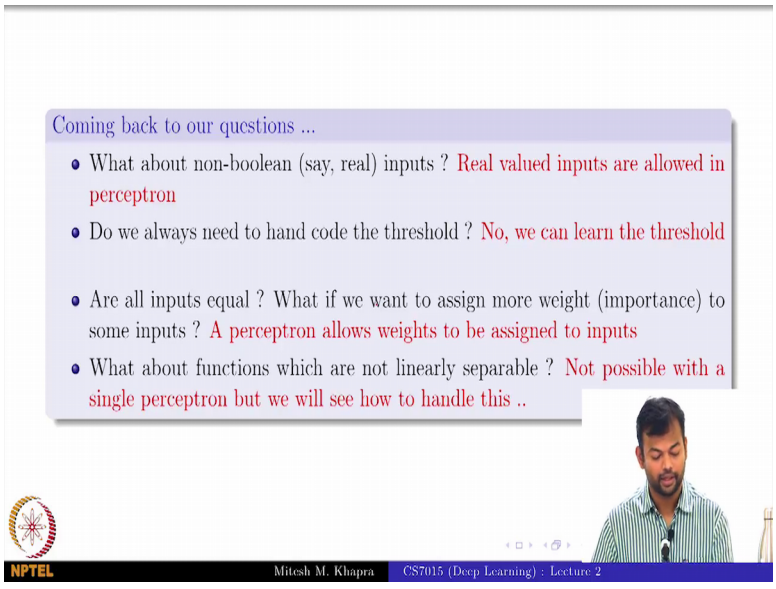
Square root of k right, you have, I mean roughly speaking you have a k here, you have a square root of k here. So, I could roughly speaking say that it is proportional to square root of k. So, as k grows what will happen to cos beta, it will grow and that is fine right, it can keep growing.

Student: (Refer Time: 13:31).

Only until one right. So, cos beta is going to be proportional to k what is k? The number of updates that you make. Now, if I were to take that degenerate case which you guys were hinting at, where that it will keep changing again and again, what will happen to k? It will keep going to infinity can that happen.

No, because cos beta will blow up right and that is not allowed. So, k has to be finite, so that cos beta stays within its limits right. Hence are we done, how many if you think we are done, how many if you are satisfy that we are done.

(Refer Slide Time: 14:29)



So, yeah. So, this says that we can only have a finite number of such k updates that we make and after that the algorithm will converge. So, we have a proof of convergence. Now, coming back to our questions this is where we had started at one point, what about non boolean inputs, so perceptron allows that. We took IMDB rating and critics rating as an input. Do we always need to hand code the threshold?

No, in our perceptron learning algorithm are all inputs equal? No we now, assign weights to input. What about functions which are not linearly separable? We still do not know. So, that is where we are headed now, not possible with a single perceptron, but we will see how to handle this.